Building a RAG System with LangChain and ChromaDB

Introduction

Retrieval-Augmented Generation (RAG) is a powerful technique that combines the capabilities of large language models with external knowledge retrieval. This notebook will walk you through building a complete RAG system using:

- LangChain: A framework for developing applications powered by language models
- ChromaDB: An open-source vector database for storing and retrieving embeddings
- OpenAI: For embeddings and language model (you can substitute with other providers)

In [1]:
```
import os
from dotenv import load_dotenv
load_dotenv()
```

Out[1]:
```
True
```

In [4]:
```
## langchain imports
from langchain_text_splitters import(
    RecursiveCharacterTextSplitter,
    CharacterTextSplitter,
    TokenTextSplitter
)


# from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader
# from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document

## vectorstores
from langchain_community.vectorstores import Chroma

## utility imports
import numpy as np
from typing import List
```

In [5]:
```
### Huggingface And OpenAI Models

from langchain_huggingface import HuggingFaceEmbeddings
# from sentence_transformers import SentenceTransformer

# Initialize a simple Embedding model(no API Key needed!)
embeddings=HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
# embeddings=SentenceTransformer(
# "sentence-transformers/all-MiniLM-L6-v2"
# )
embeddings
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 1953.94it/s]
```

Out[5]:
```
HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2', cache_folder=None, model_kwargs={}, encode_kwargs={}, query_encode_kwargs={}, multi_process=False, show_progress=False)
```

In [ ]:

In [6]:

```
# RAG Architecture Overview
print("""
RAG (Retrieval-Augmented Generation) Architecture:

1. Document Loading: Load documents from various sources
2. Document Splitting: Break documents into smaller chunks
3. Embedding Generation: Convert chunks into vector representations
4. Vector Storage: Store embeddings in ChromaDB
5. Query Processing: Convert user query to embedding
6. Similarity Search: Find relevant chunks from vector store
7. Context Augmentation: Combine retrieved chunks with query
8. Response Generation: LLM generates answer using context

Benefits of RAG:
- Reduces hallucinations
- Provides up-to-date information
- Allows citing sources
- Works with domain-specific knowledge
""")
```

```
RAG (Retrieval-Augmented Generation) Architecture:

1. Document Loading: Load documents from various sources
2. Document Splitting: Break documents into smaller chunks
3. Embedding Generation: Convert chunks into vector representations
4. Vector Storage: Store embeddings in ChromaDB
5. Query Processing: Convert user query to embedding
6. Similarity Search: Find relevant chunks from vector store
7. Context Augmentation: Combine retrieved chunks with query
8. Response Generation: LLM generates answer using context

Benefits of RAG:
- Reduces hallucinations
- Provides up-to-date information
- Allows citing sources
- Works with domain-specific knowledge
```

1. Sample Data

In [7]:

```
## create sample documents
sample_docs = [
    """
    Machine Learning Fundamentals

    Machine learning is a subset of artificial intelligence that enables systems to learn
    and improve from experience without being explicitly programmed. There are three main
    types of machine learning: supervised learning, unsupervised learning, and reinforcement
    learning. Supervised learning uses labeled data to train models, while unsupervised
    learning finds patterns in unlabeled data. Reinforcement learning learns through
    interaction with an environment using rewards and penalties.
    """,

    """
    Deep Learning and Neural Networks

    Deep learning is a subset of machine learning based on artificial neural networks.
    These networks are inspired by the human brain and consist of layers of interconnected
    nodes. Deep learning has revolutionized fields like computer vision, natural language
    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly
    effective for image processing, while Recurrent Neural Networks (RNNs) and Transformers
    excel at sequential data processing.
    """,

    """
    Natural Language Processing (NLP)

    NLP is a field of AI that focuses on the interaction between computers and human language.
    Key tasks in NLP include text classification, named entity recognition, sentiment analysis,
    machine translation, and question answering. Modern NLP heavily relies on transformer
    architectures like BERT, GPT, and T5. These models use attention mechanisms to understand
    context and relationships between words in text.
    """
]

sample_docs
```

Out[7]:

```
['\n    Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence that enables systems to learn \n    and improve from experience with
out being explicitly programmed. There are three main \n    types of machine learning: supervised learning, unsupervised learning, and reinforcement \n    learning. Su
pervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in unlabeled data. Reinforcement learning learns through \n    inter
action with an environment using rewards and penalties.\n    ',
 '\n    Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on artificial neural networks. \n    These networks are inspired by
the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fields like computer vision, natural language \n    processing, an
d speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image processing, while Recurrent Neural Networks (RNNs) and Trans
formers \n    excel at sequential data processing.\n    ',
 '\n    Natural Language Processing (NLP)\n\n    NLP is a field of AI that focuses on the interaction between computers and human language. \n    Key tasks in NLP in
clude text classification, named entity recognition, sentiment analysis, \n    machine translation, and question answering. Modern NLP heavily relies on transformer \n
architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n    context and relationships between words in text.\n    ']
```

In [10]:

```
## save sample documents to files
import tempfile
temp_dir=tempfile.mkdtemp()

for i,doc in enumerate(sample_docs):
    with open(f"{temp_dir}/doc_{i}.txt","w") as f:
        f.write(doc)

print(f"Sample document create in : {temp_dir}")
```

```
Sample document create in : C:\Users\SNIGDH~1\AppData\Local\Temp\tmptwg_y7kv
```

In [13]:

```
## save sample documents to files
import tempfile
temp_dir=tempfile.mkdtemp()

for i,doc in enumerate(sample_docs):
    with open(f'data/doc_{i}.txt',"w") as f:
        f.write(doc)
```

In [7]:
```
temp_dir
```
Out[7]:
```
'C:\\Users\\win10\\AppData\\Local\\Temp\\tmp36cdbi28'
```

2. Document Loading

In [15]:
```
from langchain_community.document_loaders import DirectoryLoader,TextLoader

# Load documents from directory
loader = DirectoryLoader(
    "data",
    glob="*.txt",
    loader_cls=TextLoader,
    loader_kwargs={'encoding': 'utf-8'}
)
documents = loader.load()

print(f"Loaded {len(documents)} documents")
print(f"\nFirst document preview:")
print(documents[0].page_content[:200] + "...")
```
```
Loaded 3 documents

First document preview:

    Machine Learning Fundamentals

    Machine learning is a subset of artificial intelligence that enables systems to learn
    and improve from experience without being explicitly programmed. Ther...
```
In [ ]:

In [16]:
```
documents
```
Out[16]:
```
[Document(metadata={'source': 'data\\doc_0.txt'}, page_content='\n    Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence tha
t enables systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised lear
ning, unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in
unlabeled data. Reinforcement learning learns through \n    interaction with an environment using rewards and penalties.\n    '),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='\n    Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based
on artificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionize
d fields like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for imag
e processing, while Recurrent Neural Networks (RNNs) and Transformers \n    excel at sequential data processing.\n    '),
 Document(metadata={'source': 'data\\doc_2.txt'}, page_content='\n    Natural Language Processing (NLP)\n\n    NLP is a field of AI that focuses on the interaction be
tween computers and human language. \n    Key tasks in NLP include text classification, named entity recognition, sentiment analysis, \n    machine translation, and q
uestion answering. Modern NLP heavily relies on transformer \n    architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n    c
ontext and relationships between words in text.\n    ')]
```

Document Splitting

In [17]:

```
# Initialize text splitter
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,  # Maximum size of each chunk
    chunk_overlap=50,  # Overlap between chunks to maintain context
    length_function=len,
    separators=["\n\n", "\n", " ", ""]  # Hierarchy of separators
)
chunks=text_splitter.split_documents(documents)

print(f"Created {len(chunks)} chunks from {len(documents)} documents")
print(f"\nChunk example:")
print(f"Content: {chunks[0].page_content[:150]}...")
print(f"Metadata: {chunks[0].metadata}")
```

```
Created 5 chunks from 3 documents

Chunk example:
Content: Machine Learning Fundamentals

Machine learning is a subset of artificial intelligence that enables systems to learn
and improve from experie...
Metadata: {'source': 'data\\doc_0.txt'}
```

In [18]:

```
chunks
```

Out[18]:

```
[Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n Machine learning is a subset of artificial intelligence that ena
bles systems to learn \n and improve from experience without being explicitly programmed. There are three main \n types of machine learning: supervised learning,
unsupervised learning, and reinforcement \n learning. Supervised learning uses labeled data to train models, while unsupervised \n learning finds patterns in unla
beled data. Reinforcement learning learns through'),
 Document(metadata={'source': 'data\\doc_0.txt'}, page_content='data. Reinforcement learning learns through \n interaction with an environment using rewards and
penalties.'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n Deep learning is a subset of machine learning based on a
rtificial neural networks. \n These networks are inspired by the human brain and consist of layers of interconnected \n nodes. Deep learning has revolutionized fiel
ds like computer vision, natural language \n processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n effective for image pro
cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n excel at sequential data processing.'),
 Document(metadata={'source': 'data\\doc_2.txt'}, page_content='Natural Language Processing (NLP)\n\n NLP is a field of AI that focuses on the interaction betwe
en computers and human language. \n Key tasks in NLP include text classification, named entity recognition, sentiment analysis, \n machine translation, and quest
ion answering. Modern NLP heavily relies on transformer \n architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n cont
ext and relationships between words in text.')]
```

Embedding Models

OPEN AI

In [12]:

```
os.environ["OPENAI_API_KEY"]=os.getenv("OPENAI_API_KEY")
```

In [13]:

```
sample_text="MAchine LEarning is fascinating"
embeddings=OpenAIEmbeddings()
embeddings
```

Out[13]:

```
OpenAIEmbeddings(client=<openai.resources.embeddings.Embeddings object at 0x000001D9674BECF0>, async_client=<openai.resources.embeddings.AsyncEmbe
ddings object at 0x000001D9674BF620>, model='text-embedding-ada-002', dimensions=None, deployment='text-embedding-ada-002', openai_api_version=None, open
ai_api_base=None, openai_api_type=None, openai_proxy=None, embedding_ctx_length=8191, openai_api_key=SecretStr('**********'), openai_organization=Non
e, allowed_special=None, disallowed_special=None, chunk_size=1000, max_retries=2, request_timeout=None, headers=None, tiktoken_enabled=True, tiktoken_mode
l_name=None, show_progress_bar=False, model_kwargs={}, skip_empty=False, default_headers=None, default_query=None, retry_min_seconds=4, retry_max_sec
onds=20, http_client=None, http_async_client=None, check_embedding_ctx_length=True)
```

In [14]:

```
vector=embeddings.embed_query(sample_text)
vector
```

Out[14]:

```
[-0.022832060232758522,
 0.011113880202174187,
 0.0084602115675807,
 -0.029978575184941292,
 -0.005767131689935923,
 0.020467406138777733,
 -0.003254685318097472,
 0.01150142028927803,
 -0.02147895283997059,
 -0.04414023086428642,
 0.006433832924813032,
 0.043851219117641451
```

0.04383121911764143,
-0.01824725605547428,
0.005139841232448816,
-0.0012406229507178068,
0.0039312392473220825,
0.03694117069244385,
0.009537442587316036,
0.004381180740892887,
-0.0077376775443553925,
-0.021649733185768127,
0.020966609939932823,
-0.0054321386851370335,
-0.04061952233314514,
-0.007409253157675266,
0.012361892499029636,
0.014647725969552994,
-0.03649451211094856,
-0.013715000823140144,
-0.002336739329621196,
0.016841599717736244,
-0.008920005522668362,
-0.019271939992904663,
-0.0407508946955204,
-0.015199478715658188,
-0.012171406298875809,
0.0001829939428716898,
0.0011905382853001356,
-0.014844780787825584,
-0.0009926626225933433,
0.017235709354281425,
0.01280198059976101,
-0.010581832379102707,
-0.025656510144472122,
-0.020585637539625168,
0.009530874900519848,
-0.0032481166999787092,
-0.01715688779950142,
-0.011462009511888027,
0.026103166863322258,
0.016105929389595985,
0.01735394261777401,
0.0035042876843363047,
-0.00533689558506012,
0.005688309669494629,
0.008519327268004417,
0.002023093868046999,
0.008906868286430836,
0.007671992760151625,
0.005103714298456907,
0.021452678367495537,
-0.018904104828834534,
-0.011547399684786797,
0.02193874679505825,
-0.013912055641412735,
0.005530666094273329,
-0.008427368476986885,
0.03809722512960434,
0.031397368758916855,
0.00854560174047947,
0.01466086320579052,
-0.010680359788239002,
-0.020559364929795265,
-0.002023093868046999,
0.02782411128282547,
-0.0031906424555927515,
-0.018956653773784637,
0.0035436986945569515,
0.012992466799914837,
0.016237299889326096,
-0.008039828389883041,
-0.0050084711983799934,
0.005300769116729498,
0.008020122535526752,
0.01353108324110508,
0.00463078310713172,
-0.01203346811234951,
-0.004128294065594673,
0.010010373778641224,
-0.0037276162765920162,
0.017800599336624146,
0.0032990225590765476,
0.011015352793037891,

0.027482548728585243,
-0.026208262890577316,
0.002996872179210186,
-0.00625648396089673,
0.018444310873746872,
0.020953472703695297,
-0.02404066175222397,
-0.02645786479115486,
0.02306852675974369,
-0.0011347060790285468,
-0.010778887197375298,
-0.015370259061455727,
-0.0015641209902241826,
0.004735879134386778,
0.010654086247086525,
0.028612328693270683,
0.007836204953491688,
-2.7454223527456634e-05,
0.03736155480146408,
-0.015028698369860649,
-0.028218219056725502,
-0.009077648632228374,
-0.01748531311750412,
-0.0017521438421681523,
-0.019166845828294754,
-0.021965021267533302,
-0.007908457890152931,
0.022385403513908386,
0.025367496535182,
0.013294617645442486,
-0.013452261686325073,
0.012020330876111984,
0.02094033546745777,
-0.011008784174919128,
-0.016565723344683647,
-0.00792816374450922,
-0.008374820463359356,
0.02363341674208641,
0.0003579825279302895,
-0.002428698120638728,
0.009563717059791088,
-0.02406693622469902,
0.020138980820775032,
0.000773028819821775,
0.005964186042547226,
-0.016565723344683647,
-0.006686719600111246,
0.014240480028092861,
0.013885781168937683,
0.009281272068619728,
-0.024960249662399292,
-0.027351180091500282,
0.009879004210233688,
0.00390168116427958,
0.02021780237555504,
0.007468369789421558,
0.0006609539850614965,
-0.004821269307285547,
-0.027140988036990166,
0.0003111820551566796,
0.020572500303387642,
0.004804848227649927,
-0.003537130309268832,
0.0244610458612442,
0.0042498111724853516,
-0.017012381926178932,
-0.013938329182565212,
-0.0009869151981547475,
-0.023528320714831352,
-0.014673999510705471,
-0.015895739197731018,
0.02267441712319851,
0.026090029627084732,
0.024119483307003975,
0.007993848994374275,
0.003635657485574484,
0.007790225557982922,
-0.000580079504288733,
0.03610040247440338,
-0.02389615587890148,
0.01087084598839283,
-0.00880834087729454,

-0.012480124831199646,
0.021899335086345673,
-0.006190799176692963,
-0.03712508827447891,
-0.01702551729977131,
0.008085807785391808,
0.018996063619852066,
0.01981055736541748,
0.01854940690100193,
-0.005803258158266544,
-0.0087754987180233,
0.009327251464128494,
-0.027009617537260056,
0.015054971911013126,
-0.0016675746301189065,
0.009852730669081211,
0.005517528858035803,
-0.02124248631298542,
-0.01067379117012024,
-0.6663073301315308,
-0.029610740020871162,
0.016145341098308563,
0.012151700444451952,
-0.007501211948692799,
0.002297328319400549,
0.0020083149429410696,
-0.010141744278371334,
0.031055806204676628,
0.02137385681271553,
0.004486276768147945,
0.008920005522668362,
-0.011389756575226784,
-0.000168112208484672,
0.007829636335372925,
-0.02806057594716549,
0.019731733947992325,
-0.05817051976919174,
-0.013386576436460018,
0.01974487118422985,
-0.01978428289294243,
0.034156132489442825,
-0.006266336422413588,
-0.0024204873479902744,
-0.013767548836767673,
0.007947869598865509,
0.012171406298875809,
0.017905695363879204,
0.0007176072103902698,
-0.007415821775794029,
-0.000804229115601629,
0.02432967536151409,
0.00021942851890344173,
-0.0039673661813139915,
0.052942004054784775,
-0.01705179177224636,
-0.00187201879452914,
0.030635423958301544,
0.013419418595731258,
0.0194164477288723,
-0.020625049248337746,
-0.013557356782257557,
-0.0034583082888275385,
0.01322893239557743,
-0.0029558190144598484,
0.006417411845177412,
0.002180737676098942 8,
0.011172995902597904,
-0.02373851090669632,
-0.02643159218132496,
0.006512654945254326,
0.008519327268004417,
-0.010811729356646538,
0.003282601246610284,
0.0032678223215043545,
0.015514765866100788,
0.01234875526279211,
-0.0046077934093773365,
0.017695503309369087,
-0.02506534568965435,
-0.016040245071053505,
0.005221947096288204,
-0.03160756081342697,

-0.011888961307704449,
-0.016013970598578453,
0.026234537363052368,
0.006213788874447346,
-0.01987624168395996,
-0.0077376775443553925,
-0.015869464725255966,
0.03733528032898903,
-0.00152963644359260 8,
0.01186925545334816,
-0.006190799176692963,
0.009635969996452332,
0.028901342302560806,
0.012723159044981003,
-0.007107102777808905,
0.009379799477756023,
0.016263574361801147,
0.008874026127159595,
-0.0017833441961556673,
-0.006013449747115374,
-0.02030976116657257,
0.02386988140642643,
-0.008204040117561817,
-0.001926208846271038,
-0.0026076892390847206,
0.0026553107891231775,
0.0028835658449679613,
0.018969791010022163,
0.020125843584537506,
-0.015212615951895714,
-0.05848580598831177,
0.011048194952309132,
-8.4056104242336e-05,
0.0024024241138249636,
0.016171615570783615,
0.009760771878063679,
-0.026983344927430153,
-0.005845953244715929,
0.004482992459088564,
0.020336035639047623,
0.020861513912677765,
0.016473764553666115,
-0.005034745205193758,
0.017367079854011536,
0.006929753813892603,
0.017275121062994003,
-0.027613919228315353,
-0.023685963824391365,
-0.0008826404227875173,
-0.023488909006118774,
0.021163664758205414,
-0.009635969996452332,
-0.026707468554377556,
0.0072384728118777275,
-0.0004084777901880443,
0.016079656779766083,
-0.029269177466630936,
0.03331536427140236,
-0.0004638993996195495,
0.01228307094424963,
-0.01865450292825699,
-0.01583005301654339,
-0.02048054151237011,
-0.004538824316114187,
-0.01868077740073204,
-0.014582040719687939,
0.006850931793451309,
-0.010844571515917778,
-0.007750814780592918,
0.0006888700881972909,
-0.022280307486653328,
0.0038688387721776962,
-0.010686928406357765,
0.015449081547558308,
-0.006818089634180069,
0.0026865112595260143,
-0.01695983298122883,
-0.024185169488191605,
0.005796689540147781,
0.022017568349838257,
0.02237226627767086,
-0.006322168745100498

-0.0003221687431004985,
-0.010916825383901596,
-0.002760406583547592,
0.01152112614363432,
0.004617646336555481,
-0.0150418346747756,
0.0014894044725224376,
-0.009031669236719608,
-0.007967574521899223,
0.025853564962744713,
-0.01025340799242258,
0.00990527868270874,
-0.012545810081064701,
-0.0013999087968841195,
-0.016578860580921173,
-0.027482548728585243,
-0.01795824244618416,
-0.004752300214022398,
-0.018641365692019463,
0.007159650791436434,
0.00024241821665782481,
-0.019455857574939728,
-0.0063911378383636475,
0.03783448413014412,
-0.041801851242780685,
-0.022424815222620964,
0.013419418595731258,
-0.0263264961540699,
0.006716277915984392,
0.022030705586075783,
-0.02044113166630268,
0.020822104066610336,
-0.01529143750667572,
-0.01155396830290556,
-0.01605338230729103,
-0.024907702580094337,
0.0014261826872825623,
0.015580451115965843,
0.0013194448547437787,
-0.007763951551169157,
0.020230939611792564,
0.017511585727334023,
0.0382285937666893,
0.01639494299888611,
-0.018260393291711807,
0.01039134617894888,
0.009511169046163559,
0.016408080235123634,
0.0043549067340791225,
-0.0018506712513044477,
0.010154880583286285,
0.014306164346635342,
-0.024474183097481728,
-0.012322481721639633,
0.009609696455299854,
0.035259637981653214,
0.0006835331441834569,
0.013872644864022732,
0.0210060216486454,
0.010562127456068993,
0.00010350498632760718,
-0.01961350254714489,
0.0011076111113652587,
-0.020690733566880226,
0.014004014432430267,
0.016775915399193764,
0.011317502707242966,
-0.004959207493811846,
-0.0014475303469225764,
-0.026181988418102264,
0.023213032633066177,
0.03946346789598465,
-0.00939293671399355,
0.0013235501246526837,
-0.02693079598248005,
0.007258178200572729,
0.0012274859473109245,
0.00011002215615008026,
0.01263120025396347,
-0.00821060873568058,
0.02260873280465603,
0.007652287371456623,

0.015028698369860649,
0.020730145275592804,
0.0024960250593721867,
-0.03906936198472977,
-0.017931969836354256,
0.014319301582872868,
0.02104543149471283,
0.005422285757958889,
0.01517320517450571,
0.0010969373397529125,
0.012007194571197033,
-0.010384777560830116,
0.03988385200500488,
-0.006246631033718586,
-0.0032678223215043545,
-0.000330066483002156,
0.018930379301309586,
-0.023055389523506165,
0.009780476801097393,
0.0049263653345406055,
0.02919035591185093,
-0.018063338473439217,
-0.025078482925891876,
0.013064720667898655,
-0.03363065421581268,
0.004269516561180353,
-0.012618063017725945,
0.005786837078630924,
-0.00042366739944554865,
-0.03268479183316231,
0.013754411600530148,
0.00021532321989070624,
0.029111534357070923,
0.00999066885560751,
-0.008591581135988235,
0.02858605608344078,
0.0037013425026088953,
-0.004266232252120972,
0.007100534159690142,
-0.012269933708012104,
-0.0008539033005945385,
-0.010036648251116276,
-0.005793405696749687,
0.007915026508271694,
-0.017866283655166626,
-0.021386994048953056,
-0.006404274608939886,
-0.026773152872920036,
0.0256959218531847,
-0.025091620162129402,
0.04046187922358513,
0.0017997653922066092,
-0.011882392689585686,
0.03410358354449272,
-0.0065323603339493275,
-0.033525556325912476,
0.01625043712556362,
0.011626222170889378,
-0.007888752967119217,
-0.026838837191462517,
-0.020664459094405174,
-0.010916825383901596,
-0.006712993606925011,
-0.002272696467116475,
0.008985689841210842,
-0.0007849342073313892,
-0.012322481721639633,
-0.01308442559093237,
-0.01837862655520439,
0.011691906489431858,
0.022661279886960983,
-0.004492844920605421,
-0.0018014075467363,
-0.02021780237555504,
-0.0006523328484036028,
0.01118613313883543,
-0.031265996396541595,
-0.01778746210038662,
0.02782411128282547,
0.0006580802728421986,
0.004056040663272142,
-0.01781373657286167,

-0.0016527954721823335,
-0.025656510144472122,
0.010581832379102707,
-0.011599947698414326,
0.01685473695397377,
-0.003954228945076466,
-0.0016101003857329488,
0.034024763852357864,
0.015028698369860649,
0.008709813468158245,
0.01709120348095894,
0.024119483307003975,
-0.011192701756954193,
-0.009616265073418617,
0.0026635215617716312,
0.01904861256480217,
0.09857985377311707,
0.011908667162060738,
-0.012361892499029636,
0.005021608434617519,
-0.0331839956343174,
-0.02832331508398056,
-0.01765609346330166,
-0.017038654536008835,
0.006134966854006052,
-0.00112239015288651,
-0.018194708973169327,
0.0004173041961621493,
0.014463808387517929,
0.005583214107900858,
0.01572495698928833,
0.006134966854006052,
0.002000104170292616,
-0.0003177505568601191,
0.007882184349000454,
-0.015935149043798447,
0.01598769798874855,
0.004959207493811846,
0.01347853522747755,
-0.00452897185459733,
0.026142578572034836,
-0.0061316825449466705,
0.0090907858684659,
0.030871888622641563,
-0.022516774013638496,
-0.03108208067715168,
0.01529143750667572,
0.028743699193000793,
-0.010102332569658756,
0.008092375472187996,
-0.012723159044981003,
0.024960249662399292,
0.002507519908249378,
0.008381389081478119,
0.02489456534385681,
-0.02306852675974369,
0.030635423958301544,
0.042169686406850815,
-0.002918050391599536,
-0.0009351883199997246,
-0.0020953472703695297,
-0.019206255674362183,
-0.0006100482423789799,
0.0331839956343174,
-0.008381389081478119,
-0.029164081439375877,
0.01150142028927803,
6.613388904952444e-06,
-0.026773152872920036,
0.0029804508667439222,
0.003839280689135194,
0.02406693622469902,
-0.018956653773784637,
0.009202450513839722,
-0.008900299668312073,
-0.015396533533930779,
-0.03071424551308155,
-0.021702280268073082,
0.004450149834156036,
-0.004092167597264051 4,
0.0137806860730052,
-0.016040245071053505,

-0.010542421601712704,
-0.011757591739296913,
-0.030109943822026253,
0.021124254912137985,
0.007304157596081495,
-0.04658370837569237,
-0.03925327956676483,
-0.019271939992904663,
0.013741274364292622,
0.01208601612597704,
0.013347165659070015,
-9.709045116323978e-05,
-0.02034917287528515,
0.01781373657286167,
-0.005356600973755121,
-0.011797002516686916,
-0.010732907801866531,
-0.022188348695635796,
0.014542629942297935,
-0.011192701756954193,
-0.011941509321331978,
-0.005215378478169441,
0.016749640926718712,
-0.006732698995620012,
0.018628228455781937,
-0.007159650791436434,
0.0256959218531847,
0.02063818648457527,
0.0013744558673352003,
0.006092271767556667,
0.009780476801097393,
0.012611495330929756,
0.011284660547971725,
-0.023515183478593826,
0.027981754392385483,
-0.020125843584537506,
-0.029269177466630936,
-0.013583631254732609,
0.014687136746942997,
0.02406693622469902,
0.010614674538373947,
0.013662452809512615,
0.012296207249164581,
-0.010509579442441463,
0.016710231080651283,
0.004781858529895544,
0.02367282658815384,
-0.005412433296442032,
0.023252444341778755,
0.014332438819110394,
0.011711612343788147,
0.01030595600605011,
-0.007166219409555197,
-0.02845468558371067,
0.002540362300351262,
-0.003507571993395686,
0.02968956157565117,
-0.00047498370986431837,
-0.0015961423050612211,
0.004693184047937393,
0.010962804779410362,
-0.013636179268360138,
-0.022267170250415802,
-0.0064141275361180305,
-0.010798592120409012,
0.03218558430671692,
-0.020730145275592804,
-0.03373574838042259,
-0.036888621747493744,
-0.017078066244721413,
-0.005717867985367775,
-0.006473243702203035,
-0.014792232774198055,
-0.003711195196956396,
-0.012585220858454704,
-0.007297588977962732,
0.012506399303674698,
-0.003586393781006336,
-0.013820096850395203,
-0.030924435704946518,
-0.002264485927298665,
0.012000623595307827,

0.012000623953507827,
0.000968307703092694,
0.0288225207477808,
-0.018588818609714508,
0.039279550313949585,
-0.012421009130775928,
-0.013938329182565212,
-0.014503219164907932,
-0.002095347270369529 7,
0.002443477045744657 5,
0.00491322809830308,
0.055700767785310745,
0.030766792595386505,
0.05115537717938423,
-0.010240270756185055,
0.025722194463014603,
0.012171406298875809,
-0.00448956061154604,
-0.02393556572496891,
0.007389547768980265,
-0.005330326966941357,
-0.004752300214022398,
0.011462009511888027,
0.014424397610127926,
-0.00660789804533124,
0.010752612724900246,
-0.011245249770581722,
0.005987175740301609,
0.003428750205785036,
0.0014335722662508488,
-0.015646135434508324,
-0.034156132489442825,
-0.013885781168937683,
-0.013570494018495083,
0.02111111767590046,
0.01548849232494831,
0.02167600765824318,
-0.02161032147705555,
0.009274703450500965,
0.03683607652783394,
0.006335305515676737,
0.0030543464235961437,
0.020519953221082687,
0.021268760785460472,
-0.012355323880910873,
-0.010962804779410362,
0.023580867797136307,
0.0035765410866588354,
-0.0369674451649189,
-0.005639045964926481,
-0.026208262890577316,
-0.01284796092659235,
0.03300007805228233,
0.016066519543528557,
0.02267441712319851,
0.031003259122371674,
1.2450618669390678e-05,
-0.0219124723225832,
-0.001778417848981917,
-0.01685473695397377,
-0.009445483796298504,
-0.005041313823312521,
-0.03662588447332382,
-0.005350032821297646,
-0.01765609346330166,
0.0016314479289576411,
-0.013636179268360138,
0.000755376007873565,
0.001146200927905738 4,
-0.027114713564515114,
0.012453851290047169,
-0.03862270340323448,
-0.010548990219831467,
0.02107170596718788,
0.029637012630701065,
0.044035136699676514,
-0.002817880827933555,
0.022845197468996048,
0.010259976610541344,
0.013235501013696194,
0.01622416265308857,
0.014450671151280403,

-0.006128398235887289,
4.782679388881661e-05,
0.02319989539682865,
0.007501211948692799,
-0.02482888102531433,
-0.029269177466630936,
0.008696677163243294,
-0.010351935401558876,
-0.014332438819110394,
-0.019587228074669838,
0.004850827623158693,
0.0030182197224348783,
0.010043216869235039,
-0.02333126589655876,
-0.007113671395927668,
-0.022293444722890854,
0.019771145656704903,
-0.022280307486653328,
0.012375029735267162,
-0.01534398552030325,
-0.004052756354212761,
-0.0006346800364553928,
0.008000416681170464,
-0.02030976116657257,
0.0007586602587252855,
-0.015672409906983376,
0.012493262067437172,
0.027377452701330185,
-0.0039805029518902 3,
-0.0012956340797245502,
0.005120135378092527,
-0.01027311384677887,
0.02729863114655018,
-0.0019048611866310239,
0.0013194448547437787,
0.0005004365812055767,
0.01785314828157425,
-0.01661827228963375,
-0.015422807075083256,
-0.0012833181535825133,
0.024316538125276566,
-0.004614362027496 0995,
-0.017038654536008835,
-0.00511028291657567,
0.017511585727334023,
0.019928788766264915,
-0.017038654536008835,
-0.011849550530314445,
0.0015164994401857257,
0.004594656638801098,
-0.004834406543523073,
0.015383396297693253,
0.01987624168395996,
-0.01265090610831976,
0.011416030116379261,
-0.013609904795885086,
-0.015882601961493492,
-0.001147843082435429,
0.007816499099135399,
0.0187858734279871,
-0.02480260655283928,
-0.01265090610831976,
0.008381389081478119,
0.004680046811699867,
0.008164629340171814,
0.014713411219418049,
-0.024920839816331863,
-0.014989287592470646,
0.03683607652783394,
-0.01067379117012024,
0.006696572527289 3906,
0.01759040728211403,
0.011041626334190369,
-0.019337626174092293,
0.01266404241323471,
0.001101863686926663,
-0.015527903102338314,
0.002083852421492338,
-0.013596767559647 56,
0.007665424142032862,
-0.004900091327726841,
0.005051166284829378,

0.006404274608939886,
-0.0004610256874002516,
-0.019074885174632072,
0.012815117835998535,
0.014871054328978062,
0.0040494720451533794,
-0.004581519868224859,
0.000983630889095366,
0.012493262067437172,
0.00509386183694005,
-0.0072319041937589645,
0.0036652158014476 3,
-0.020769555121660233,
0.004660341423004866,
-0.003845849074423313,
-0.005632477346807718,
-0.002461540512740612,
-0.04563784599304199,
0.002578131156042218,
-0.00905794370919466,
0.016237299889326096,
-0.00526792649179697,
-0.0004901733482256532,
0.0023482341784983873,
0.0029246187768876553,
-0.021951884031295776,
-0.007245040964335203,
-0.028927616775035858,
-0.007534054573625326,
0.01364931557327509,
0.005872227251529694,
0.020191529765725136,
-0.012657474726438522,
-0.009892141446471214,
-0.010759181343019009,
-0.005061019212007523,
-0.009839593432843685,
-0.036284320056438446,
-0.007317294366657734,
-0.028664877638220787,
0.03155501186847687,
0.006213788874447346,
-0.011606516316533089,
-0.018969791010022163,
-6.650592695223168e-05,
-0.025170441716909 41,
0.0137806860730052,
-0.006095556076616049,
0.015120657160878181,
0.019061749801039696,
0.011310934089124203,
-0.014752821996808052,
0.0200470220297575,
-0.021925609558820724,
0.012184543535113335,
-0.011606516316533089,
0.003678352804854512,
0.0028063859790563583,
0.005471549462527037,
-0.00917617604136467,
-0.01384637039154768,
-0.040672071278095245,
0.0021511793602257967,
0.01622416265308857,
-0.011356913484632969,
0.0036947738844901323,
0.0015789000317454338,
-0.012618063017725945,
-0.0003017398703377694,
0.01774805225431919,
-0.011764160357415676,
0.005865658633410931,
-0.004765437450259924,
0.006433832924813032,
-0.015869464725255966,
-0.01294648740440607,
0.0054551283828914165,
-0.010476736351847649,
-0.0044797081500291824,
-0.013406282290816307,
0.002144610974937677 4,
0.002543646609410643 6,

-0.002689795335754752,
-0.012184543535113335,
0.0015050045913085341,
0.0017669230001047254,
-0.004098735749721527,
0.028402136638760567,
-0.0023236023262143135,
-0.011888961307704449,
0.030871888622641563,
0.0007561970851384103,
0.00840766355395317,
-0.009504600428044796,
0.0021446109749376774,
-0.0004134041373617947,
-0.011994057334959507,
0.01215170044451952,
-0.01795824244618416,
0.0009975889697670937,
-0.0244610458612442,
0.006240062415599823,
-0.009051375091075897,
0.005323758814483881,
-0.00897912122309208,
0.0016848169034346938,
-0.016000833362340927,
0.0022070116829127073,
-0.00826315674930811,
-0.008703244850039482,
0.006545497104525566,
0.0018424605950713158,
-0.004151283763349056,
-0.01628984697163105,
-0.010877414606511593,
0.017275121062994003,
-0.008236882276833057,
-0.006240062415599823,
0.012696885503828526,
-0.009616265073418617,
-0.00945862103253603,
0.010660653933882713,
0.018129024654626846,
-0.002187306061387062,
0.0033482862636446953,
0.22364383935928345,
-0.036152951419353485,
-0.0017997653922066092,
0.0022201486863195896,
-0.0013169816229492426,
0.012782275676727295,
0.007494643796235323,
0.006085703149437904,
0.0013982666423544288,
0.0023925714194774628,
0.022083252668380737,
-0.0056784567423164845,
-0.04611077904701233,
-0.00438774935901165,
0.013288049027323723,
4.556887870421633e-05,
-0.0275613721460104,
-0.009268134832382202,
0.007034849375486374,
-0.01944272220134735,
0.016211025416851044,
0.006463391240686178,
-0.027929207310080528,
-0.03497062623500824,
-0.0039673661813139915,
0.0021676006726920605,
-0.011113880202174187,
0.01208601612597704,
-0.005527381785213947,
0.0014212563401088119,
-0.008834615349769592,
-0.007573465351015329,
0.0019754725508391857,
-0.008479916490614414,
-0.004676762502640486,
-0.011980920098721981,
0.016316121444106102,
-0.003451739903539419,
0.027324905619023523,

0.027324903619025 23,
0.014595177955925465,
0.01831294223666191,
-0.01798451691865921,
0.013806959614157677,
-0.024881428107619286,
0.002640531864017248,
0.012171406298875809,
...]

Hugging Face

In [19]:
Initialize a simple Embedding model(no API Key needed!)
embeddings=HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
print(embeddings)

Loading weights: 100%|| 103/103 [00:00<00:00, 3776.74it/s]
model_name='sentence-transformers/all-MiniLM-L6-v2' cache_folder=None model_kwargs={} encode_kwargs={} query_encode_kwargs={} multi_process=False s
how_progress=False

In [20]:
sample_text="MAchine LEarning is fascinating"
embeddings=embeddings.embed_query(sample_text)
embeddings

Out[20]:
[-0.04763009026646614,
 -0.09523377567529678,
 0.08622033149003983,
 0.008249782025814056,
 0.012193558737635612,
 -0.050495196133852005,
 -0.007320730481296778,
 -0.05885452404618263,
 -0.03901902958750725,
 -0.028660964220762253,
 -0.08570092916488647,
 0.07921122014522552,
 0.00468570739030838,
 -0.02150493673980236,
 -0.07088708132505417,
 0.014649443328380585,
 -0.01347882766276598,
 -0.015381773933768272,
 -0.08215576410293579,
 -0.12226202338933945,
 -0.0034171852748841047,
 0.01304476149380207,
 -0.008553941734135151,
 0.007963702082633972,
 0.02498506009578705,
 0.018824271857738495,
 0.06268897652626038,
 0.019554147496819496,
 0.04684524983167648,
 -0.06655009835958481,
 0.0034160162322223186,
 0.06802240759134293,
 -0.011118346825242043,
 0.030044380575418472,
 -0.09648731350898743,
 0.027357490733265877,
 0.009005188941955566,
 0.017103202641010284,
 0.04554043710231781,
 0.006371472962200642,
 -0.005477484315633774,
 -0.04323989897966385,
 0.03637200966477394,
 -0.0049531301483511925,
 0.11256805807352066,
 0.10895713418722153,
 -0.058884162455797195,
 -0.08978395909070969,
 0.01417580433189869,
 -0.004923671018332243,
 -0.12424919754266739,
 -0.04150596261024475,
 -0.03020506352186203,

-0.009999031201004982,
-0.08218608796596527,
0.018560079857707024,
0.052757058292627335,
-0.021651865914463997,
-0.0035149662289768457,
-0.018109627068042755,
0.018467623740434647,
-0.07166921347379684,
-0.0766029804944992,
-5.186835187487304e-05,
0.08697870373725891,
-0.022721294313669205,
-0.0016885988879948854,
0.03807207569479942,
-0.018713636323809624,
-0.0386103019118309,
-0.04819657653570175,
0.12013337016105652,
0.01635143905878067,
0.11162298917770386,
0.07147170603275299,
0.03678681701421738,
0.048580121248960495,
0.04590976983308792,
0.09048043936491013,
0.04488284885883331,
0.019618211314082146,
-0.06778345257043839,
0.00888123083859682,
0.040763355791568756,
0.06069207936525345,
-0.12897832691669464,
-0.04271581023931503,
-0.0032787565141916275,
-0.009827842935919762,
0.0005153662641532719,
-0.0030698252376168966,
0.0051942686550319195,
0.0317864790558815,
-0.09975674003362656,
-0.014281682670116425,
0.03796965628862381,
0.0005382410599850118,
-0.05450158566236496,
0.013058596290647984,
0.09638986736536026,
-0.08256544917821884,
0.02337964065372944,
0.03767920285463333,
0.018936170265078545,
0.04331546276807785,
-0.011235186830163002,
0.05404801666736603,
0.013554343022406101,
0.15480855107307434,
-0.11832716315984726,
-0.0211191289126873,
0.06041416525840759,
0.0323149710893631,
-0.06339867413043976,
0.01684175245463848,
-0.008745059370994568,
0.028255155310034752,
0.02347773313522339,
-0.00500714685767889,
0.06584012508392334,
-0.03957684710621834,
0.004732244182378054,
-0.08012419939041138,
0.040118731558322906,
0.029002873227000237,
-0.07128506898880005,
-0.05239695310592651 4,
-1.851412969765711e-33,
-0.0224587582051754,
-0.05324174463748932,
0.0374113954603672,
0.03622003644704819,
0.058388855308294296,
-0.10132131725549698,
-0.04294372349077493

-0.042943723499774 93,
-0.07153549045324326,
-0.0022056661546230316,
0.00991623755544424,
-0.03496255353093147,
0.09678725898265839,
0.014809886924922466,
0.015479432418942451,
0.013518028892576694,
-0.02324460633099079,
-0.05069880932569504,
0.06064721569418907,
-0.010043333284556866,
-0.06408016383647919,
0.02763615921139717,
0.016566477715969086,
0.05769830942153931,
0.02126978524029255,
-0.04161033034324646,
0.03467780724167824,
0.026389559730887413,
-0.024245835840702057,
0.0639873519539833,
0.037308644503355026,
-0.027789214625954628,
0.004934582393616438,
-0.0850943773984909,
0.011615953408181667,
0.005482211709022522,
0.010303880088031292,
-0.005995346233248711,
-0.024837203323841095,
0.08113263547420502,
0.027461815625429153,
0.024977434426546097,
-0.05153562128543854,
0.026391249150037766,
-0.07550927996635437,
-0.03513886034488678,
0.026748988777399063,
0.05017092078924179,
-0.07794103026390076,
-0.01852072961628437,
-0.07365947216749191,
-0.06532637774944305,
-0.020507659763097763,
-0.03154166042804718,
-0.012163510546088219,
0.042026977986097336,
0.0584181472659111,
0.03136267140507698,
0.012510489672422409,
-0.005927486810833216,
-0.003578329458832741,
0.009319174103438854,
0.0626571774482727,
0.014164145104587078,
0.044353045523166656,
-0.032045893371105194,
0.05018295720219612,
0.049104347825050354,
-0.0048691583797335625,
0.056328028440475464,
0.04199821874499321,
-0.006348013877868652,
0.032427698373794556,
0.03214237093925476,
-0.09874063730239868,
-0.021463915705680847,
-0.02502269856631756,
0.04340203106403351,
-0.030517209321260452,
-0.02650829218327999,
0.005576716270297766,
-0.057877954095602036,
-0.034901779145002365,
0.03966328501701355,
-0.047214385122060776,
0.015531034208834171,
0.000900997081771493,
0.013943289406597614,
-0.08569415658712387,

-0.07319215685129166,
0.038371741771698,
-0.05251685157418251,
-0.040434058755636215,
0.06753028929233551,
-0.003971809055656195,
-0.0728810653090477,
1.5378931980380523e-33,
-0.13390976190567017,
-0.012025962583720684,
0.01882568560540676,
0.10021147131919861,
-0.019922597333788872,
0.0065399883314967155,
-0.07442920655012131,
0.02666759118437767,
-0.06070571765303612,
0.03277286887168884,
0.05751313269138336,
0.03343759849667549,
0.0813278928399086,
0.015304438769817352,
0.01394287496805191,
0.033637214452028275,
-0.023404870182275772,
0.015400411561131477,
-0.04267460107803345,
0.00570281594991684,
0.007098241243511438,
0.10701968520879745,
-0.0436333604156971,
-0.01397420559078455,
0.028677593916654587,
0.008382171392440796,
-0.08829445391893387,
-0.001931157661601901,
0.023776954039931297,
0.03156699240207672,
-0.07297558337450027,
0.004157829564064741,
-0.05567466467618942,
-0.03396712616086006,
-0.06280216574668884,
0.053711388260126114,
0.023145386949181557,
-0.02265065722167492,
0.013378278352320194,
0.1064750999212265,
0.029187314212322235,
0.009746589697897434,
-0.031947121024131775,
-0.027531422674655914,
-0.013691205531358719,
-0.05204359441995621,
0.004957919474691153,
0.02288341335952282,
0.03773296996951103,
-0.020572222769260406,
0.026852916926145554,
0.04887457564473152,
-0.04705943912267685,
-0.07500672340393066,
-0.04644506424665451,
0.0208851657807827,
-0.039417773485183716,
0.0025468822568655014,
0.07679826021194458,
0.05375231057405472,
-0.11899691820144653,
-0.008447710424661636,
0.009597843512892723,
0.04310064762830734,
0.008501382544636726,
-0.006365731358528137,
-0.02668660879135132,
0.04639572650194168,
-0.08796430379152298,
-0.08304373174905777,
0.03635283559560776,
0.0317482054233551,
-0.05327317491173744,
0.06381669640541077,

-0.04489888995885849,
-0.025313690304756165,
0.011110114865005016,
-0.0467047393321991,
-0.07605022192001343,
-0.01842224784195423,
0.0671527311205864,
-0.08439595997333527,
0.035344772040843964,
0.04648497700691223,
0.031017664819955826,
0.10328135639429092,
0.052219100296497345,
-0.036084819585084915,
-0.011852308176457882,
-0.017376946285367012,
-0.04490082338452339,
0.03258572518825531,
-0.02754071168601513,
0.06971318274736404,
-0.014728407375514507,
-1.1417436773797363e-08,
0.011337337084114552,
0.0066170175559818745,
0.05255453288555145,
-0.061447903513908386,
0.09359519928693771,
0.0229098629206419,
-0.02789226546883583,
0.11478246748447418,
-0.10289320349693298,
0.014907597564160824,
0.08839994668960571,
-0.020931823179125786,
-0.0012170724803581834,
0.10379936546087265,
0.053682342171669006,
0.0180397666990757,
0.03834952414035797,
0.04144167900085449,
-0.02341323345899582,
0.04420550912618637,
0.06359720975160599,
0.04357866942882538,
0.05021534860134125,
-0.02138371765613556,
0.02935398370027542,
-0.02486340142786503,
-0.030586279928684235,
0.019653694704174995,
0.060944896191358566,
0.000472828047350049,
-0.07703299820423126,
0.09105832874774933,
0.01881439797580242,
0.015357572585344315,
0.07740549743175507,
0.060757238417863846,
-0.008226633071899414,
-0.06321871280670166,
-0.04141616448760033,
-0.04288209602236748,
-0.02191021479666233,
0.06281843036413193,
-0.06046108901500702,
0.03939468413591385,
-0.010597520507872105,
0.028410444036126137,
0.040635522454977036,
-0.09052857011556625,
0.018001442775130272,
0.07328018546104431,
0.05442802235484123,
-0.008709531277418137,
0.0733216255903244,
0.060961633920669556,
0.09512174874544144,
-0.003931817598640919,
-0.006277403794229031,
-0.03624698892235756,
-0.048207614570856094,
0.1187029704451561,

0.07818420231342316,
0.07516173273324966,
-0.0214520413428545,
-0.05170662701129913]

Intilialize the ChromaDB Vector Store And Stores the chunks in Vector Representation

In [21]:
chunks

Out[21]:
[Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n Machine learning is a subset of artificial intelligence that ena bles systems to learn \n and improve from experience without being explicitly programmed. There are three main \n types of machine learning: supervised learning, unsupervised learning, and reinforcement \n learning. Supervised learning uses labeled data to train models, while unsupervised \n learning finds patterns in unla beled data. Reinforcement learning learns through'),
Document(metadata={'source': 'data\\doc_0.txt'}, page_content='data. Reinforcement learning learns through \n interaction with an environment using rewards and penalties.'),
Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n Deep learning is a subset of machine learning based on a rtificial neural networks. \n These networks are inspired by the human brain and consist of layers of interconnected \n nodes. Deep learning has revolutionized fiel ds like computer vision, natural language \n processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n effective for image pro cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n excel at sequential data processing.'),
Document(metadata={'source': 'data\\doc_2.txt'}, page_content='Natural Language Processing (NLP)\n\n NLP is a field of AI that focuses on the interaction betwe en computers and human language. \n Key tasks in NLP include text classification, named entity recognition, sentiment analysis, \n machine translation, and quest ion answering. Modern NLP heavily relies on transformer \n architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n cont ext and relationships between words in text.')]

Open AI

In [16]:
```
## Create a Chromdb vector store
persist_directory="./chroma_db"

## Initialize Chromadb with Open AI embeddings
vectorstore=Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(),
    persist_directory=persist_directory,
    collection_name="rag_collection"

)

print(f"Vector store created with {vectorstore._collection.count()} vectors")
print(f"Persisted to: {persist_directory}")
```

```
Failed to send telemetry event ClientStartEvent: capture() takes 1 positional argument but 3 were given
Failed to send telemetry event ClientCreateCollectionEvent: capture() takes 1 positional argument but 3 were given
Vector store created with 10 vectors
Persisted to: ./chroma_db
```

Hugging Face

In [22]:
```
## Create a Chromdb vector store
persist_directory="./chroma_db"

## Initialize Chromadb with Open AI embeddings
vectorstore=Chroma.from_documents(
    documents=chunks,
    embedding=HuggingFaceEmbeddings(),
    persist_directory=persist_directory,
    collection_name="rag_collection"

)

print(f"Vector store created with {vectorstore._collection.count()} vectors")
print(f"Persisted to: {persist_directory}")
```

d:\Souvick\AI_Engeenering\RAG\KrisNayak\RAG\.venv\Lib\site-packages\huggingface_hub\file_download.py:150: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\Snigdha Mazumdar\.cache\huggingface\hub\models--sentence-transformers--all-mpnet-base-v2. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
  warnings.warn(message)
d:\Souvick\AI_Engeenering\RAG\KrisNayak\RAG\.venv\Lib\site-packages\huggingface_hub\file_download.py:150: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\Snigdha Mazumdar\.cache\huggingface\hub. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
  warnings.warn(message)
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 922.11it/s]
Vector store created with 5 vectors
Persisted to: ./chroma_db

Test Similarity Search

In [24]:
```
query="What are the types of machine learning?"

similar_docs=vectorstore.similarity_search(query,k=5)
similar_docs
```

Out[24]:
```
[Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence that enables systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised learning, unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in unlabeled data. Reinforcement learning learns through'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on artificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fields like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image processing, while Recurrent Neural Networks (RNNs) and Transformers'),
 Document(metadata={'source': 'data\\doc_0.txt'}, page_content='data. Reinforcement learning learns through \n    interaction with an environment using rewards and penalties.'),
 Document(metadata={'source': 'data\\doc_2.txt'}, page_content='Natural Language Processing (NLP)\n\n    NLP is a field of AI that focuses on the interaction between computers and human language. \n    Key tasks in NLP include text classification, named entity recognition, sentiment analysis, \n    machine translation, and question answering. Modern NLP heavily relies on transformer \n    architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n    context and relationships between words in text.'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n    excel at sequential data processing.')]
```

In [25]:
```
query="what is NLP?"

similar_docs=vectorstore.similarity_search(query,k=3)
similar_docs
```

Out[25]:
```
[Document(metadata={'source': 'data\\doc_2.txt'}, page_content='Natural Language Processing (NLP)\n\n    NLP is a field of AI that focuses on the interaction between computers and human language. \n    Key tasks in NLP include text classification, named entity recognition, sentiment analysis, \n    machine translation, and question answering. Modern NLP heavily relies on transformer \n    architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n    context and relationships between words in text.'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on artificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fields like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image processing, while Recurrent Neural Networks (RNNs) and Transformers'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n    excel at sequential data processing.')]
```

In [26]:
```
query="what is Deep Learning?"

similar_docs=vectorstore.similarity_search(query,k=3)
similar_docs
```

Out[26]:
```
[Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on artificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fields like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image processing, while Recurrent Neural Networks (RNNs) and Transformers'),
 Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence that enables systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised learning, unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in unlabeled data. Reinforcement learning learns through'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n    excel at sequential data processing.')]
```

In [28]:

```
print(f"Query: {query}")
print(f"\nTop {len(similar_docs)} similar chunks:")
for i, doc in enumerate(similar_docs):
    print(f"\n--- Chunk {i+1} ---")
    print(doc.page_content[:200] + "...")
    print(f"Source: {doc.metadata.get('source', 'Unknown')}")
```

```
Query: what is Deep Learning?

Top 3 similar chunks:

--- Chunk 1 ---
Deep Learning and Neural Networks

    Deep learning is a subset of machine learning based on artificial neural networks.
    These networks are inspired by the human brain and consist of layers of i...
Source: data\doc_1.txt

--- Chunk 2 ---
Machine Learning Fundamentals

    Machine learning is a subset of artificial intelligence that enables systems to learn
    and improve from experience without being explicitly programmed. There are...
Source: data\doc_0.txt

--- Chunk 3 ---
Neural Networks (RNNs) and Transformers
    excel at sequential data processing....
Source: data\doc_1.txt
```

Advanced Similarity Search With Scores

In [29]:

```
results_scores=vectorstore.similarity_search_with_score(query,k=3)
results_scores
```

Out[29]:

```
[(Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on
artificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fi
elds like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image pr
ocessing, while Recurrent Neural Networks (RNNs) and Transformers'),
  0.4973113536834717),
 (Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence that en
ables systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised learning
, unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in unla
beled data. Reinforcement learning learns through'),
  0.8716006278991699),
 (Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n    excel at sequential data processing.'),
  1.1829462051391602)]
```

Understanding Similarity Scores

The similarity score represents how closely related a document chunk is to your query. The scoring depends on the distance metric used:

ChromaDB default: Uses L2 distance (Euclidean distance)

- Lower scores = MORE similar (closer in vector space)
- Score of 0 = identical vectors
- Typical range: 0 to 2 (but can be higher)

Cosine similarity (if configured):

- Higher scores = MORE similar
- Range: -1 to 1 (1 being identical)

In [ ]:

In [ ]:

ChatCompletion(id='gen-1792492823-Y6SVE5QzfMtVlG2R2A73', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='Hello! I\'m Nemotron, a large language model developed by NVIDIA. Here\'s a quick overview of how I work:\n\n**What I am:** I\'m trained on a vast and diverse collection of text from the internet, books, articles, and other sources. I don\'t "think" or have consciousness—I process patterns in language to generate helpful, coherent responses.\n\n**How I operate:** When you give me a prompt, I analyze the context and predict the most relevant and useful continuation based on the patterns I learned during training. I don\'t have personal experiences, beliefs, or real-time awareness of the world.\n\n**What I can help with:** answering questions, brainstorming ideas, writing/editing text, coding, translation, summarizing, tutoring, creative projects, and much more.\n\n**Important limits:** \n- I can occasionally generate incorrect, misleading, or "hallucinated" information, so it\'s always good to verify critical facts.\n- I don\'t have access to your personal data, private accounts, or the live internet unless specifically equipped with browsing tools.\n- I don\'t retain memory of past conversations unless the platform I\'m running on preserves context session-by-session.\n\nJust type naturally, ask your question, or tell me what you\'re working on—and I\'ll do my best to help! What\'s on your mind?', refusal=None, role='assistant', annotations=None, audio=None, function_call=None, tool_calls=None, reasoning='Here\'s a thinking process:\n\n1. **Analyze User Input:**\n - User says: "Hello! How do you work?"\n - This is a greeting + a question about my functionality/operation\n - I need to respond helpfully, explaining how I work as an AI, without overpromising or giving misleading info\n\n2. **Identify Key Topics to Cover:**\n - What I am (large language model)\n - How I\'m trained (on diverse text data)\n - What I can do (answer questions, help with tasks, creative work, etc.)\n - Limitations (no real-time awareness, no personal experiences, can make mistakes, etc.)\n - How users can interact with me (ask questions, give prompts, etc.)\n - Safety/ethics note (briefly)\n\n3. **Structure the Response:**\n - Friendly greeting\n - Core explanation of what I am and how I work\n - Capabilities\n - Limitations\n - How to interact with me\n - Offer to help with specific tasks\n\n Keep it concise but comprehensive. Avoid jargon where possible, or explain it.\n\n4. **Draft - Mental Refinement:**\n - "Hello! I\'m Nemotron, a language model trained by NVIDIA researchers." -> Standard, good.\n - "I work by processing the text you input, predicting the most relevant and helpful response based on patterns I learned from a vast amount of text data during training." -> Accurate.\n - "I can help with things like answering questions, brainstorming, writing, coding, translation, and much more." -> Good list.\n - "I don\'t have personal experiences, consciousness, or real-time awareness of the world. I also can\'t access your personal data or the internet unless specifically equipped with tools." -> Important disclaimer.\n - "I strive to be accurate, but I can sometimes make mistakes or hallucinate, so it\'s always good to verify critical information." -> Responsible AI note.\n - "Just type or speak naturally, and I\'ll do my best to help! What\'s on your mind?" -> Friendly close.\n\n Check against guidelines: No overclaiming, clear about limitations, helpful, concise. Looks solid.\n\n5. **Final Output Generation:** (matches the refined draft)□', reasoning_details=[{'type': 'reasoning.text', 'text': 'Here\'s a thinking process:\n\n1. **Analyze User Input:**\n - User says: "Hello! How do you work?"\n - This is a greeting + a question about my functionality/operation\n - I need to respond helpfully, explaining how I work as an AI, without overpromising or giving misleading info\n\n2. **Identify Key Topics to Cover:**\n - What I am (large language model)\n - How I\'m trained (on diverse text data)\n - What I can do (answer questions, help with tasks, creative work, etc.)\n - Limitations (no real-time awareness, no personal experiences, can make mistakes, etc.)\n - How users can interact with me (ask questions, give prompts, etc.)\n - Safety/ethics note (briefly)\n\n3. **Structure the Response:**\n - Friendly greeting\n - Core explanation of what I am and how I work\n - Capabilities\n - Limitations\n - How to interact with me\n - Offer to help with specific tasks\n\n Keep it concise but comprehensive. Avoid jargon where possible, or explain it.\n\n4. **Draft - Mental Refinement:**\n - "Hello! I\'m Nemotron, a language model trained by NVIDIA researchers." -> Standard, good.\n - "I work by processing the text you input, predicting the most relevant and helpful response based on patterns I learned from a vast amount of text data during training." -> Accurate.\n - "I can help with things like answering questions, brainstorming, writing, coding, translation, and much more." -> Good list.\n - "I don\'t have personal experiences, consciousness, or real-time awareness of the world. I also can\'t access your personal data or the internet unless specifically equipped with tools." -> Important disclaimer.\n - "I strive to be accurate, but I can sometimes make mistakes or hallucinate, so it\'s always good to verify critical information." -> Responsible AI note.\n - "Just type or speak naturally, and I\'ll do my best to help! What\'s on your mind?" -> Friendly close.\n\n Check against guidelines: No overclaiming, clear about limitations, helpful, concise. Looks solid.\n\n5. **Final Output Generation:** (matches the refined draft)□', 'format': 'unknown', 'index': 0}]), native_finish_reason='stop')], created=1792492823, model='nvidia/nemotron-3.5-lightning:free', object='chat.completion', metadata=None, moderation=None, service_tier=None, system_fingerprint=None, usage=CompletionUsage(completion_tokens=764, prompt_tokens=23, total_tokens=787, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=None, audio_tokens=0, reasoning_tokens=486, rejected_prediction_tokens=None, text_tokens=None, image_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cache_write_tokens=0, cached_tokens=0, image_tokens=None, text_tokens=None, video_tokens=0), cost=0, is_byok=False, cost_details={'upstream_inference_cost': 0, 'upstream_inference_prompt_cost': 0, 'upstream_inference_completions_cost': 0}), provider='Nvidia')

Initialize LLM, RAG Chain, Prompt Template,Query the RAG system

In [22]:

```
from langchain_openai import ChatOpenAI

llm=ChatOpenAI(
    model_name="gpt-3.5-turbo"
)
```

Free LLMs from Open Router

In [61]:

```
import os
from dotenv import load_dotenv
from openai import OpenAI
from langchain_openai import ChatOpenAI

load_dotenv(override=True)
api_key = os.getenv("OPENROUTER_API_KEY")
if not api_key:
    raise ValueError("OPENROUTER_API_KEY was not loaded")

llm = ChatOpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
    model_name="nvidia/nemotron-3.5-lightning:free"
)

# response = llm.chat.completions.create(
#     # model="openai/gpt-4o-mini",
#     model="nvidia/nemotron-3.5-lightning:free",
#     messages=[{"role": "user", "content": "Hello! what are your capabilities?"}],
# )
# print(response)
```

In [44]:

```
test_response=llm.invoke("What is Large Language Models")
test_response
```

Out[44]:

AIMessage(content='**Large Language Models (LLMs)** are advanced artificial intelligence systems trained on massive amounts of text data to understand, generate, translate, and manipulate human language. The "large" refers both to the sheer volume of training data (often billions of words from books, articles, websites, and code) and to the model\'s parameter count, which can range from hundreds of millions to trillions.\n\n### How They Work\n- **Architecture:** Most LLMs are built on the **Transformer** architecture, which uses self-attention mechanisms to weigh the importance of different words in a sentence relative to one another, regardless of their position.\n- **Training Goal:** They\'re trained to predict the next word (or token) in a sequence. Through this process, they learn grammar, facts, reasoning patterns, coding syntax, context, and even some forms of logic.\n- **Scale:** Modern LLMs have billions or trillions of adjustable parameters, requiring massive computational resources for both training and inference.\n\n### Core Capabilities\n- **Text generation:** Writing essays, stories, emails, code, poetry, etc.\n- **Summarization:** Condensing long articles or documents.\n- **Translation:** Converting text between languages.\n- **Question answering:** Providing factual or inferential answers from a given context or internal knowledge.\n- **Coding assistance:** Writing, debugging, and explaining code in various programming languages.\n- **Classification & extraction:** Sentiment analysis, entity extraction, topic labeling, etc.\n\n### Well-Known Examples\n- **OpenAI:** GPT-3.5, GPT-4, GPT-4o\n- **Anthropic:** Claude 3 series\n- **Google:** Gemini (1.0, 1.5, Flash, Pro)\n- **Meta:** Llama 2, Llama 3\n- **Mistral:** Mixtral, Mistral Large\n- **Chinese models:** Zhipu ChatGLM, Baidu Ernie, etc.\n\n### Common Use Cases\n- Chatbots & virtual assistants\n- Content creation & copywriting\n- Code completion & software development aids\n- Research & summarization tools\n- Translation & localization\n- Internal knowledge bases & customer support automation\n\n### Important Limitations\n- **Hallucinations:** Models can confidently state false or made-up information.\n- **Lack of true understanding:** They don\'t "know" what they\'re saying; they predict based on patterns, not meaning or consciousness.\n- **Bias & safety concerns:** Training data can reflect societal biases, and models may generate harmful, biased, or inappropriate content without careful guarding.\n- **Context windows:** They have limited "memory" per conversation; beyond that, earlier context is lost.\n- **Resource intensity:** Training and running LLMs requires significant electricity, specialized hardware (GPUs/TPUs), and financial investment.\n- **Privacy & copyright:** Training data may include copyrighted material, and input prompts can sometimes be retained or exposed.\n\n### Bottom Line\nLLMs are powerful pattern-matching tools that can handle a wide variety of language tasks with impressive fluency. They\'re not sentient, nor do they "understand" language the way humans do, but they\'ve become foundational infrastructure for many AI-powered products, research assistants, and automation workflows. Their usefulness grows with better prompting, grounding in verified data, and integration with other tools (search, databases, code interpreters, etc.).', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 1150, 'prompt_tokens': 21, 'total_tokens': 1171, 'completion_tokens_details': {'accepted_prediction_tokens': None, 'audio_tokens': 0, 'reasoning_tokens': 443, 'rejected_prediction_tokens': None, 'text_tokens': None, 'image_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cache_write_tokens': 0, 'cached_tokens': 0, 'image_tokens': None, 'text_tokens': None, 'video_tokens': 0}, 'cost': 0, 'is_byok': False, 'cost_details': {'upstream_inference_cost': 0, 'upstream_inference_prompt_cost': 0, 'upstream_inference_completions_cost': 0}}, 'model_provider': 'openai', 'model_name': 'nvidia/nemotron-3.5-lightning:free', 'system_fingerprint': None, 'id': 'gen-1792493233-9sVMsbZuS67HQ0WP2igi', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--01a158ec-2a3f-79e0-a6d0-b4f7a2a36847-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 21, 'output_tokens': 1150, 'total_tokens': 1171, 'input_token_details': {'audio': 0, 'cache_read': 0, 'cache_creation': 0}, 'output_token_details': {'audio': 0, 'reasoning': 443}})

In [62]:

```
from langchain.chat_models.base import init_chat_model

# # llm=init_chat_model("openai:gpt-3.5-turbo")
# llm=init_chat_model("nvidia/nemotron-3.5-lightning:free")
# #llm=init_chat_model("groq:")
# llm


llm = init_chat_model(
    model="nvidia/nemotron-3.5-lightning:free",
    model_provider="openai",
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
)
llm
```

Out[62]:
```
ChatOpenAI(metadata={'lc_versions': {'langchain-core': '1.6.4', 'langchain': '1.4.2', 'langchain-openai': '1.6.4'}}, client=<openai.resources.chat.completions.completions.Completions object at 0x000001EB000F33F0>, async_client=<openai.resources.chat.completions.completions.AsyncCompletions object at 0x000001EB75F7EBE0>, root_client=<openai.OpenAI object at 0x000001EB7562FBB0>, root_async_client=<openai.AsyncOpenAI object at 0x000001EB7562F950>, model_name='nvidia/nemotron-3.5-lightning:free', model_kwargs={}, openai_api_key=SecretStr('**********'), openai_api_base='https://openrouter.ai/api/v1', stream_chunk_timeout=120.0)
```
In [63]:
```
llm.invoke("What is AI")
```
Out[63]:
```
AIMessage(content="**Artificial Intelligence (AI)** refers to computer systems or machines that can perform tasks that typically require human intelligence. These tasks include learning, reasoning, problem-solving, understanding language, recognizing patterns, making decisions, and perceiving the environment.\n\n### How It Works\nMost modern AI doesn't follow explicit step-by-step instructions for every scenario. Instead, it uses **machine learning (ML)**: algorithms learn patterns and rules from large datasets. A powerful subset of ML is **deep learning**, which uses artificial neural networks with many layers to model complex relationships in data (like images, speech, or text).\n\n### Two Main Categories\n| Type | Description | Status |\n|------|-------------|--------|\n| **Narrow (or Weak) AI** | Designed for a specific task or set of tasks. It doesn't have consciousness or general understanding. | ✅ This is what we have today (voice assistants, recommendation engines, image classifiers, autonomous driving, medical diagnosis tools, large language models like me, etc.) |\n| **General (or Strong) AI** | Hypothetical AI that could understand, learn, and apply intelligence across any task at a human-level or beyond. It would have adaptability, self-awareness, and consciousness. | ❌ Not yet achieved; remains a subject of research and debate |\n\n### Common Everyday Examples\n- Voice assistants (Siri, Alexa, Google Assistant)\n- Recommendation systems (Netflix, Spotify, YouTube)\n- Spam filters and fraud detection\n- Language models (chatbots, translation, summarization)\n- Facial recognition & photo tagging\n- Autonomous vehicles & drones\n- Medical imaging analysis & drug discovery\n\n### Benefits & Considerations\n**Benefits:** Automates repetitive work, accelerates discovery, improves accuracy in diagnostics, enables personalization, handles dangerous or data-heavy tasks.\n\n**Challenges/Ethics:** Algorithmic bias, privacy concerns, deepfakes, job displacement, accountability, and the need for responsible governance. Most experts emphasize developing AI that is transparent, fair, and aligned with human values.\n\nIn short: AI is a broad field of computer science focused on building systems capable of intelligent behavior. Today, it's mostly **narrow AI** that powers tools we use daily, with ongoing research toward safer, more capable, and more general forms in the future.", additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 955, 'prompt_tokens': 19, 'total_tokens': 974, 'completion_tokens_details': {'accepted_prediction_tokens': None, 'audio_tokens': 0, 'reasoning_tokens': 468, 'rejected_prediction_tokens': None, 'text_tokens': None, 'image_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cache_write_tokens': 0, 'cached_tokens': 0, 'image_tokens': None, 'text_tokens': None, 'video_tokens': 0}, 'cost': 0, 'is_byok': False, 'cost_details': {'upstream_inference_cost': 0, 'upstream_inference_prompt_cost': 0, 'upstream_inference_completions_cost': 0}}, 'model_provider': 'openai', 'model_name': 'nvidia/nemotron-3.5-lightning:free', 'system_fingerprint': None, 'id': 'gen-1792562153-pJ14mvZxIeqI5Kulq2pE', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--01a15d07-c7c5-7dc3-8d73-309dbe566598-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 19, 'output_tokens': 955, 'total_tokens': 974, 'input_token_details': {'audio': 0, 'cache_read': 0, 'cache_creation': 0}, 'output_token_details': {'audio': 0, 'reasoning': 468}})
```

Modern RAG Chain

In [64]:
```
# from langchain.chains import create_retrieval_chain
# from langchain_classic.chains import create_history_aware_retriever
from langchain_classic.chains import (
    create_retrieval_chain,
    create_history_aware_retriever,
)
from langchain_core.prompts import ChatPromptTemplate
from langchain_classic.chains.combine_documents import create_stuff_documents_chain
# from langchain.chains.combine_documents import create_stuff_documents_chain
```
In [65]:
```
## Convert vector store to retriever
retriever=vectorstore.as_retriever(
    search_kwarg={"k":3} ## Retrieve top 3 relevant chunks
)
retriever
```
Out[65]:
```
VectorStoreRetriever(tags=['Chroma', 'HuggingFaceEmbeddings'], vectorstore=<langchain_community.vectorstores.chroma.Chroma object at 0x000001EB0273D010>, search_kwargs={})
```
In [66]:
```
## Create a prompt template
from langchain_core.prompts import ChatPromptTemplate
system_prompt="""You are an assistant for question-answering tasks.
Use the following pieces of retrieved context to answer the question.
If you don't know the answer, just say that you don't know.
Use three sentences maximum and keep the answer concise.

Context: {context}"""

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}")
])
```
In [67]:
```
prompt
```

Out[67]:
ChatPromptTemplate(input_variables=['context', 'input'], input_types={}, partial_variables={}, messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=['context'], input_types={}, partial_variables={}, template="You are an assistant for question-answering tasks. \nUse the following pieces of retrieved context to answer the question. \nIf you don't know the answer, just say that you don't know. \nUse three sentences maximum and keep the answer concise.\n\nContext: {context}"), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'], input_types={}, partial_variables={}, template='{input}'), additional_kwargs={})])

What is create_stuff_documents_chain?

create_stuff_documents_chain creates a chain that "stuffs" (inserts) all retrieved documents into a single prompt and sends it to the LLM. It's called "stuff" because it literally stuffs all the documents into the context window at once.

In [68]:
```
### Create a document chain
from langchain_classic.chains.combine_documents import create_stuff_documents_chain

document_chain=create_stuff_documents_chain(llm,prompt)
document_chain
```

Out[68]:
RunnableBinding(bound=RunnableBinding(bound=RunnableAssign(mapper={
  context: RunnableLambda(format_docs)
}), kwargs={}, config={'run_name': 'format_inputs'}, config_factories=[])
| ChatPromptTemplate(input_variables=['context', 'input'], input_types={}, partial_variables={}, messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=['context'], input_types={}, partial_variables={}, template="You are an assistant for question-answering tasks. \nUse the following pieces of retrieved context to answer the question. \nIf you don't know the answer, just say that you don't know. \nUse three sentences maximum and keep the answer concise.\n\nContext: {context}"), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'], input_types={}, partial_variables={}, template='{input}'), additional_kwargs={})])
| ChatOpenAI(metadata={'lc_versions': {'langchain-core': '1.6.4', 'langchain': '1.4.2', 'langchain-openai': '1.6.4'}}, client=<openai.resources.chat.completions.completions.Completions object at 0x000001EB000F33F0>, async_client=<openai.resources.chat.completions.completions.AsyncCompletions object at 0x000001EB75F7EBE0>, root_client=<openai.OpenAI object at 0x000001EB7562FBB0>, root_async_client=<openai.AsyncOpenAI object at 0x000001EB7562F950>, model_name='nvidia/nemotron-3.5-lightning:free', model_kwargs={}, openai_api_key=SecretStr('**********'), openai_api_base='https://openrouter.ai/api/v1', stream_chunk_timeout=120.0)
| StrOutputParser(), kwargs={}, config={'run_name': 'stuff_documents_chain'}, config_factories=[])

This chain:

- Takes retrieved documents
- "Stuffs" them into the prompt's {context} placeholder
- Sends the complete prompt to the LLM
- Returns the LLM's response

What is create_retrieval_chain?

create_retrieval_chain is a function that combines a retriever (which fetches relevant documents) with a document chain (which processes those documents with an LLM) to create a complete RAG pipeline.

In [69]:
```
### Create The Final RAG Chain
# Modern Import Path
from langchain_classic.chains import create_retrieval_chain
rag_chain=create_retrieval_chain(retriever,document_chain)
rag_chain
```

Out[69]:
RunnableBinding(bound=RunnableAssign(mapper={
  context: RunnableBinding(bound=RunnableLambda(lambda x: x['input'])
           | VectorStoreRetriever(tags=['Chroma', 'HuggingFaceEmbeddings'], vectorstore=<langchain_community.vectorstores.chroma.Chroma object at 0x000001EB0273D010>, search_kwargs={}), kwargs={}, config={'run_name': 'retrieve_documents'}, config_factories=[])
})
| RunnableAssign(mapper={
    answer: RunnableBinding(bound=RunnableBinding(bound=RunnableAssign(mapper={
              context: RunnableLambda(format_docs)
            }), kwargs={}, config={'run_name': 'format_inputs'}, config_factories=[])
            | ChatPromptTemplate(input_variables=['context', 'input'], input_types={}, partial_variables={}, messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=['context'], input_types={}, partial_variables={}, template="You are an assistant for question-answering tasks. \nUse the following pieces of retrieved context to answer the question. \nIf you don't know the answer, just say that you don't know. \nUse three sentences maximum and keep the answer concise.\n\nContext: {context}"), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'], input_types={}, partial_variables={}, template='{input}'), additional_kwargs={})])
            | ChatOpenAI(metadata={'lc_versions': {'langchain-core': '1.6.4', 'langchain': '1.4.2', 'langchain-openai': '1.6.4'}}, client=<openai.resources.chat.completions.completions.Completions object at 0x000001EB000F33F0>, async_client=<openai.resources.chat.completions.completions.AsyncCompletions object at 0x000001EB75F7EBE0>, root_client=<openai.OpenAI object at 0x000001EB7562FBB0>, root_async_client=<openai.AsyncOpenAI object at 0x000001EB7562F950>, model_name='nvidia/nemotron-3.5-lightning:free', model_kwargs={}, openai_api_key=SecretStr('**********'), openai_api_base='https://openrouter.ai/api/v1', stream_chunk_timeout=120.0)
            | StrOutputParser(), kwargs={}, config={'run_name': 'stuff_documents_chain'}, config_factories=[])
  }), kwargs={}, config={'run_name': 'retrieval_chain'}, config_factories=[])

In [ ]:

In [70]:
```
response=rag_chain.invoke({"input":"What is Deep LEarning"})
```
In [33]:
```
response
```
Out[33]:
{'input': 'What is Deep LEarning',
'context': [Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n Deep learning is a subset of machine learning b ased on artificial neural networks. \n These networks are inspired by the human brain and consist of layers of interconnected \n nodes. Deep learning has revoluti onized fields like computer vision, natural language \n processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n effective for image processing, while Recurrent Neural Networks (RNNs) and Transformers'),
Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n Deep learning is a subset of machine learning based on a rtificial neural networks. \n These networks are inspired by the human brain and consist of layers of interconnected \n nodes. Deep learning has revolutionized fiel ds like computer vision, natural language \n processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n effective for image pro cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n Machine learning is a subset of artificial intelligence that en ables systems to learn \n and improve from experience without being explicitly programmed. There are three main \n types of machine learning: supervised learning , unsupervised learning, and reinforcement \n learning. Supervised learning uses labeled data to train models, while unsupervised \n learning finds patterns in unla beled data. Reinforcement learning learns through'),
Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n Machine learning is a subset of artificial intelligence that en ables systems to learn \n and improve from experience without being explicitly programmed. There are three main \n types of machine learning: supervised learning , unsupervised learning, and reinforcement \n learning. Supervised learning uses labeled data to train models, while unsupervised \n learning finds patterns in unla beled data. Reinforcement learning learns through')],
'answer': 'Deep learning is a subset of machine learning that relies on artificial neural networks inspired by the human brain, consisting of interconnected layers of no des. It has significantly advanced fields such as computer vision, natural language processing, and speech recognition. Convolutional Neural Networks (CNNs), Rec urrent Neural Networks (RNNs), and Transformers are commonly used architectures in deep learning for tasks like image processing and sequential data analysis.'}
In [71]:
```
response['answer']
```
Out[71]:
'Deep learning is a subset of machine learning based on artificial neural networks inspired by the human brain, consisting of layers of interconnected nodes. It has rev olutionized fields like computer vision, natural language processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly effective for ima ge processing, while RNNs and Transformers excel at sequential data processing.'
In [35]:
```
# Function to query the modern RAG system
def query_rag_modern(question):
    print(f"Question: {question}")
    print("-" * 50)

    # Using create_retrieval_chain approach
    result = rag_chain.invoke({"input": question})

    print(f"Answer: {result['answer']}")
    print("\nRetrieved Context:")
    for i, doc in enumerate(result['context']):
        print(f"\n--- Source {i+1} ---")
        print(doc.page_content[:200] + "...")

    return result

# Test queries
test_questions = [
    "What are the three types of machine learning?",
    "What is deep learning and how does it relate to neural networks?",
    "What are CNNs best used for?"
]

for question in test_questions:
    result = query_rag_modern(question)
    print("\n" + "="*80 + "\n")
```
```
Question: What are the three types of machine learning?
--------------------------------------------------
Answer: The three main types of machine learning are supervised learning, unsupervised learning, and reinforcement learning. Supervised learning uses labeled data to train models, unsupervised learning finds patterns in unlabeled data, and reinforcement learning learns through feedback from the environment.

Retrieved Context:

--- Source 1 ---
Machine Learning Fundamentals

   Machine learning is a subset of artificial intelligence that enables systems to learn
   and improve from experience without being explicitly programmed. There are...
```

--- Source 2 ---
Machine Learning Fundamentals

Machine learning is a subset of artificial intelligence that enables systems to learn
and improve from experience without being explicitly programmed. There are...

--- Source 3 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks.
These networks are inspired by the human brain and consist of layers of i...

--- Source 4 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks.
These networks are inspired by the human brain and consist of layers of i...

==========

Question: What is deep learning and how does it relate to neural networks?

Answer: Deep learning is a subset of machine learning that relies on artificial neural networks inspired by the human brain. These neural networks consist of intercon nected nodes organized in layers. Deep learning has significantly advanced fields such as computer vision, natural language processing, and speech recognition.

Retrieved Context:

--- Source 1 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks.
These networks are inspired by the human brain and consist of layers of i...

--- Source 2 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks.
These networks are inspired by the human brain and consist of layers of i...

--- Source 3 ---
Neural Networks (RNNs) and Transformers
excel at sequential data processing....

--- Source 4 ---
Neural Networks (RNNs) and Transformers
excel at sequential data processing....

==========

Question: What are CNNs best used for?

Answer: CNNs are particularly effective for image processing due to their ability to capture spatial hierarchies in data.

Retrieved Context:

--- Source 1 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks.
These networks are inspired by the human brain and consist of layers of i...

--- Source 2 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks.
These networks are inspired by the human brain and consist of layers of i...

--- Source 3 ---
Neural Networks (RNNs) and Transformers
excel at sequential data processing....

--- Source 4 ---
Neural Networks (RNNs) and Transformers
excel at sequential data processing....

==========

Create RAG Chain Alternative - Using LCEL (LangChain Expression Language)

In [38]:

```
# Even more flexible approach using LCEL
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
```

In [39]:

```
# Create a custom prompt
custom_prompt = ChatPromptTemplate.from_template("""Use the following context to answer the question.
If you don't know the answer based on the context, say you don't know.
Provide specific details from the context to support your answer.

Context:
{context}

Question: {question}

Answer:""")
custom_prompt
```

Out[39]:

```
ChatPromptTemplate(input_variables=['context', 'question'], input_types={}, partial_variables={}, messages=[HumanMessagePromptTemplate(prompt=PromptTempl
ate(input_variables=['context', 'question'], input_types={}, partial_variables={}, template="Use the following context to answer the question. \nIf you don't know the
answer based on the context, say you don't know.\nProvide specific details from the context to support your answer.\n\nContext:\n{context}\n\nQuestion: {question
}\n\nAnswer:"), additional_kwargs={})])
```

In [40]:

```
retriever
```

Out[40]:

```
VectorStoreRetriever(tags=['Chroma', 'OpenAIEmbeddings'], vectorstore=<langchain_community.vectorstores.chroma.Chroma object at 0x000001D9674BFB60>, searc
h_kwargs={})
```

In [41]:

```
## Format the output documents for the prompt
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)
```

In [62]:

```
## Build the chain ussing LCEL

rag_chain_lcel=(
    {
        "context":retriever | format_docs,
        "question": RunnablePassthrough()
     }
    | custom_prompt
    | llm
    | StrOutputParser()
)

rag_chain_lcel
```

Out[62]:

```
{
  context: VectorStoreRetriever(tags=['Chroma', 'OpenAIEmbeddings'], vectorstore=<langchain_community.vectorstores.chroma.Chroma object at 0x000001D9674BFB
60>, search_kwargs={})
           | RunnableLambda(format_docs),
  question: RunnablePassthrough()
}
| ChatPromptTemplate(input_variables=['context', 'question'], input_types={}, partial_variables={}, messages=[HumanMessagePromptTemplate(prompt=PromptTem
plate(input_variables=['context', 'question'], input_types={}, partial_variables={}, template="Use the following context to answer the question. \nIf you don't know t
he answer based on the context, say you don't know.\nProvide specific details from the context to support your answer.\n\nContext:\n{context}\n\nQuestion: {questi
on}\n\nAnswer:"), additional_kwargs={})])
| ChatOpenAI(client=<openai.resources.chat.completions.completions.Completions object at 0x000001D96AA93110>, async_client=<openai.resources.chat.completio
ns.completions.AsyncCompletions object at 0x000001D96AA92FD0>, root_client=<openai.OpenAI object at 0x000001D96AB19810>, root_async_client=<openai.As
yncOpenAI object at 0x000001D96AB196E0>, model_kwargs={}, openai_api_key=SecretStr('**********'))
| StrOutputParser()
```

In [63]:

```
response=rag_chain_lcel.invoke("What is Deep Learning")
response
```

Out[63]:

```
'Deep learning is a subset of machine learning based on artificial neural networks. These networks consist of layers of interconnected nodes, inspired by the human b
rain, and have revolutionized fields like computer vision, natural language processing, and speech recognition.'
```

In [64]:

```
retriever.get_relevant_documents("What is Deep Learning")
```

Out[64]:
[Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n Deep learning is a subset of machine learning based on a rtificial neural networks. \n These networks are inspired by the human brain and consist of layers of interconnected \n nodes. Deep learning has revolutionized fiel ds like computer vision, natural language \n processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n effective for image pro cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n Deep learning is a subset of machine learning based on a rtificial neural networks. \n These networks are inspired by the human brain and consist of layers of interconnected \n nodes. Deep learning has revolutionized fiel ds like computer vision, natural language \n processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n effective for image pro cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n Machine learning is a subset of artificial intelligence that ena bles systems to learn \n and improve from experience without being explicitly programmed. There are three main \n types of machine learning: supervised learning, unsupervised learning, and reinforcement \n learning. Supervised learning uses labeled data to train models, while unsupervised \n learning finds patterns in unla beled data. Reinforcement learning learns through'),
Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n Machine learning is a subset of artificial intelligence that ena bles systems to learn \n and improve from experience without being explicitly programmed. There are three main \n types of machine learning: supervised learning, unsupervised learning, and reinforcement \n learning. Supervised learning uses labeled data to train models, while unsupervised \n learning finds patterns in unla beled data. Reinforcement learning learns through')]
In [65]:
```
# Query using the LCEL approach - Fixed version
def query_rag_lcel(question):
    print(f"Question: {question}")
    print("-" * 50)

    # Method 1: Pass string directly (when using RunnablePassthrough)
    answer = rag_chain_lcel.invoke(question)
    print(f"Answer: {answer}")

    # Get source documents separately if needed
    docs = retriever.get_relevant_documents(question)
    print("\nSource Documents:")
    for i, doc in enumerate(docs):
        print(f"\n--- Source {i+1} ---")
        print(doc.page_content[:200] + "...")
```
In [66]:
```
# Test LCEL chain
print("Testing LCEL Chain:")
query_rag_lcel("What are the key concepts in reinforcement learning?")
```
```
Testing LCEL Chain:
Question: What are the key concepts in reinforcement learning?
--------------------------------------------------
Answer: The key concepts in reinforcement learning are interaction with an environment, using rewards and penalties for learning. This involves the system learning through trial and error, receiving feedback in the form of rewards and penalties to adjust its behavior.

Source Documents:

--- Source 1 ---
data. Reinforcement learning learns through
    interaction with an environment using rewards and penalties....

--- Source 2 ---
data. Reinforcement learning learns through
    interaction with an environment using rewards and penalties....

--- Source 3 ---
Machine Learning Fundamentals

    Machine learning is a subset of artificial intelligence that enables systems to learn
    and improve from experience without being explicitly programmed. There are...

--- Source 4 ---
Machine Learning Fundamentals

    Machine learning is a subset of artificial intelligence that enables systems to learn
    and improve from experience without being explicitly programmed. There are...
```
In [67]:
```
query_rag_lcel("What is machine learning?")
```

Question: What is machine learning?

Answer: Machine learning is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. It consi sts of three main types: supervised learning, unsupervised learning, and reinforcement learning.

Source Documents:

--- Source 1 ---
Machine Learning Fundamentals

Machine learning is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. There are...

--- Source 2 ---
Machine Learning Fundamentals

Machine learning is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. There are...

--- Source 3 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks. These networks are inspired by the human brain and consist of layers of i...

--- Source 4 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks. These networks are inspired by the human brain and consist of layers of i...

In [69]:
query_rag_lcel("What is depe learning?")

Question: What is depe learning?

Answer: Deep learning is a subset of machine learning based on artificial neural networks. These networks consist of layers of interconnected nodes and are inspired by the human brain. Deep learning has revolutionized fields such as computer vision, natural language processing, and speech recognition. Convolutional Neural Net works (CNNs) are especially effective for image processing in deep learning.

Source Documents:

--- Source 1 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks. These networks are inspired by the human brain and consist of layers of i...

--- Source 2 ---
Deep Learning and Neural Networks

Deep learning is a subset of machine learning based on artificial neural networks. These networks are inspired by the human brain and consist of layers of i...

--- Source 3 ---
Machine Learning Fundamentals

Machine learning is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. There are...

--- Source 4 ---
Machine Learning Fundamentals

Machine learning is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. There are...

Add New Documents To Existing Vector Store

In [73]:
vectorstore

Out[73]:
<langchain_community.vectorstores.chroma.Chroma at 0x1d9674bfb60>

In [74]:

```
# Add new documents to the existing vector store
new_document = """
Reinforcement Learning in Detail

Reinforcement learning (RL) is a type of machine learning where an agent learns to make
decisions by interacting with an environment. The agent receives rewards or penalties
based on its actions and learns to maximize cumulative reward over time. Key concepts
in RL include: states, actions, rewards, policies, and value functions. Popular RL
algorithms include Q-learning, Deep Q-Networks (DQN), Policy Gradient methods, and
Actor-Critic methods. RL has been successfully applied to game playing (like AlphaGo),
robotics, and autonomous systems.
"""
```

In [75]:
```
new_document
```

Out[75]:
```
'\nReinforcement Learning in Detail\n\nReinforcement learning (RL) is a type of machine learning where an agent learns to make \ndecisions by interacting with an en
vironment. The agent receives rewards or penalties \nbased on its actions and learns to maximize cumulative reward over time. Key concepts \nin RL include: states, a
ctions, rewards, policies, and value functions. Popular RL \nalgorithms include Q-learning, Deep Q-Networks (DQN), Policy Gradient methods, and \nActor-Critic met
hods. RL has been successfully applied to game playing (like AlphaGo), \nrobotics, and autonomous systems.\n'
```

In [76]:
```
chunks
```

Out[76]:
```
[Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence that ena
bles systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised learning,
unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in unla
beled data. Reinforcement learning learns through'),
 Document(metadata={'source': 'data\\doc_0.txt'}, page_content='data. Reinforcement learning learns through \n    interaction with an environment using rewards and
penalties.'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on a
rtificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fiel
ds like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image pro
cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
 Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Neural Networks (RNNs) and Transformers \n    excel at sequential data processing.'),
 Document(metadata={'source': 'data\\doc_2.txt'}, page_content='Natural Language Processing (NLP)\n\n    NLP is a field of AI that focuses on the interaction betwe
en computers and human language. \n    Key tasks in NLP include text classification, named entity recognition, sentiment analysis, \n    machine translation, and quest
ion answering. Modern NLP heavily relies on transformer \n    architectures like BERT, GPT, and T5. These models use attention mechanisms to understand \n    cont
ext and relationships between words in text.')]
```

In [77]:
```
new_doc=Document(
    page_content=new_document,
    metadata={"source": "manual_addition", "topic": "reinforcement_learning"}
)
```

In [79]:
```
new_doc
```

Out[79]:
```
Document(metadata={'source': 'manual_addition', 'topic': 'reinforcement_learning'}, page_content='\nReinforcement Learning in Detail\n\nReinforcement learning (RL)
is a type of machine learning where an agent learns to make \ndecisions by interacting with an environment. The agent receives rewards or penalties \nbased on its ac
tions and learns to maximize cumulative reward over time. Key concepts \nin RL include: states, actions, rewards, policies, and value functions. Popular RL \nalgorith
ms include Q-learning, Deep Q-Networks (DQN), Policy Gradient methods, and \nActor-Critic methods. RL has been successfully applied to game playing (like Alpha
Go), \nrobotics, and autonomous systems.\n')
```

In [81]:
```
## split the documents
new_chunks=text_splitter.split_documents([new_doc])
new_chunks
```

Out[81]:
```
[Document(metadata={'source': 'manual_addition', 'topic': 'reinforcement_learning'}, page_content='Reinforcement Learning in Detail\n\nReinforcement learning (RL) i
s a type of machine learning where an agent learns to make \ndecisions by interacting with an environment. The agent receives rewards or penalties \nbased on its ac
tions and learns to maximize cumulative reward over time. Key concepts \nin RL include: states, actions, rewards, policies, and value functions. Popular RL \nalgorith
ms include Q-learning, Deep Q-Networks (DQN), Policy Gradient methods, and \nActor-Critic methods. RL has been'),
 Document(metadata={'source': 'manual_addition', 'topic': 'reinforcement_learning'}, page_content='methods, and \nActor-Critic methods. RL has been successfully a
pplied to game playing (like AlphaGo), \nrobotics, and autonomous systems.')]
```

In [82]:
```
### Add new documents to vectorstore
vectorstore.add_documents(new_chunks)
```

Out[82]:
```
['b9b7448a-f68b-4593-a297-84e5697793f6',
 '4669dce8-2bbb-4d08-a11c-1f8b9ab2c01b']
```

In [83]:

```
print(f"Added {len(new_chunks)} new chunks to the vector store")
print(f"Total vectors now: {vectorstore._collection.count()}")
```

```
Added 2 new chunks to the vector store
Total vectors now: 12
```

In [84]:

```
## query with the updated vector
new_question="What are the keys concepts in reinforcement learning"
result=query_rag_lcel(new_question)
result
```

```
Question: What are the keys concepts in reinforcement learning
----------------------------------------------------
Answer: The key concepts in reinforcement learning include states, actions, rewards, policies, and value functions.

Source Documents:

--- Source 1 ---
Reinforcement Learning in Detail

Reinforcement learning (RL) is a type of machine learning where an agent learns to make
decisions by interacting with an environment. The agent receives rewards or p...

--- Source 2 ---
data. Reinforcement learning learns through
    interaction with an environment using rewards and penalties....

--- Source 3 ---
data. Reinforcement learning learns through
    interaction with an environment using rewards and penalties....

--- Source 4 ---
methods, and
Actor-Critic methods. RL has been successfully applied to game playing (like AlphaGo),
robotics, and autonomous systems....
```

Advanced Rag Techniques- Conversational Memory

Understanding Conversational Memory in RAG Conversational memory enables RAG systems to maintain context across multiple interactions. This is crucial for:

Follow-up questions that reference previous answers Pronoun resolution (e.g., "it", "they", "that") Context-dependent queries that build on prior discussion Natural dialogue flow where users don't repeat context

Key Challenge: Traditional RAG retrieves documents based only on the current query, missing important context from the conversation. For example:

User: "Tell me about Python" Bot: explains Python programming language User: "What are its main libraries?" ← "its" refers to Python, but retriever doesn't know this

Solution: The modern approach uses a two-step process:

Query Reformulation: Transform context-dependent questions into standalone queries Context-Aware Retrieval: Use the reformulated query to fetch relevant documents

- create_history_aware_retriever: Makes the retriever understand conversation context
- MessagesPlaceholder: Placeholder for chat history in prompts
- HumanMessage/AIMessage: Structured message types for conversation history

In [85]:

```
from langchain.chains import create_history_aware_retriever
from langchain_core.prompts import MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
```

In [86]:

```
## create a prompt that includes the chat history
contextualize_q_system_prompt = """Given a chat history and the latest user question
which might reference context in the chat history, formulate a standalone question
which can be understood without the chat history. Do NOT answer the question,
just reformulate it if needed and otherwise return it as is."""

contextualize_q_prompt = ChatPromptTemplate.from_messages([
    ("system", contextualize_q_system_prompt),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
])
```

In [87]:

```
## create history aware retriever
history_aware_retriever = create_history_aware_retriever(
    llm, retriever, contextualize_q_prompt
)
history_aware_retriever
```

Out[87]:

```
RunnableBinding(bound=RunnableBranch(branches=[(RunnableLambda(lambda x: not x.get('chat_history', False)), RunnableLambda(lambda x: x['input'])
| VectorStoreRetriever(tags=['Chroma', 'OpenAIEmbeddings'], vectorstore=<langchain_community.vectorstores.chroma.Chroma object at 0x000001D9674BFB60>, search_kwargs={})], default=ChatPromptTemplate(input_variables=['chat_history', 'input'], input_types={'chat_history': list[typing.Annotated[typing.Union[typing.Annotated[langchain_core.messages.ai.AIMessage, Tag(tag='ai')], typing.Annotated[langchain_core.messages.human.HumanMessage, Tag(tag='human')], typing.Annotated[langchain_core.messages.chat.ChatMessage, Tag(tag='chat')], typing.Annotated[langchain_core.messages.system.SystemMessage, Tag(tag='system')], typing.Annotated[langchain_core.messages.function.FunctionMessage, Tag(tag='function')], typing.Annotated[langchain_core.messages.tool.ToolMessage, Tag(tag='tool')], typing.Annotated[langchain_core.messages.ai.AIMessageChunk, Tag(tag='AIMessageChunk')], typing.Annotated[langchain_core.messages.human.HumanMessageChunk, Tag(tag='HumanMessageChunk')], typing.Annotated[langchain_core.messages.chat.ChatMessageChunk, Tag(tag='ChatMessageChunk')], typing.Annotated[langchain_core.messages.system.SystemMessageChunk, Tag(tag='SystemMessageChunk')], typing.Annotated[langchain_core.messages.function.FunctionMessageChunk, Tag(tag='FunctionMessageChunk')], typing.Annotated[langchain_core.messages.tool.ToolMessageChunk, Tag(tag='ToolMessageChunk')]], FieldInfo(annotation=NoneType, required=True, discriminator=Discriminator(discriminator=<function _get_type at 0x000001D9367196C0>, custom_error_type=None, custom_error_message=None, custom_error_context=None))]}, partial_variables={}, messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[], input_types={}, partial_variables={}, template='Given a chat history and the latest user question \nwhich might reference context in the chat history, formulate a standalone question \nwhich can be understood without the chat history. Do NOT answer the question, \njust reformulate it if needed and otherwise return it as is.'), additional_kwargs={}), MessagesPlaceholder(variable_name='chat_history'), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['input'], input_types={}, partial_variables={}, template='{input}'), additional_kwargs={})])
| ChatOpenAI(client=<openai.resources.chat.completions.completions.Completions object at 0x000001D96AA93110>, async_client=<openai.resources.chat.completions.completions.AsyncCompletions object at 0x000001D96AA92FD0>, root_client=<openai.OpenAI object at 0x000001D96AB19810>, root_async_client=<openai.AsyncOpenAI object at 0x000001D96AB196E0>, model_kwargs={}, openai_api_key=SecretStr('**********'))
| StrOutputParser()
| VectorStoreRetriever(tags=['Chroma', 'OpenAIEmbeddings'], vectorstore=<langchain_community.vectorstores.chroma.Chroma object at 0x000001D9674BFB60>, search_kwargs={}), kwargs={}, config={'run_name': 'chat_retriever_chain'}, config_factories=[])
```

In [88]:

```
# Create a new document chain with history
qa_system_prompt = """You are an assistant for question-answering tasks.
Use the following pieces of retrieved context to answer the question.
If you don't know the answer, just say that you don't know.
Use three sentences maximum and keep the answer concise.

Context: {context}"""

qa_prompt = ChatPromptTemplate.from_messages([
    ("system", qa_system_prompt),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
])

question_answer_chain = create_stuff_documents_chain(llm, qa_prompt)

# Create conversational RAG chain
conversational_rag_chain = create_retrieval_chain(
    history_aware_retriever,
    question_answer_chain
)
print("Conversational RAG chain created!")
```

```
Conversational RAG chain created!
```

In [89]:

```
chat_history=[]
# First question
result1 = conversational_rag_chain.invoke({
    "chat_history": chat_history,
    "input": "What is machine learning?"
})
print(f"Q: What is machine learning?")
print(f"A: {result1['answer']}")
```

```
Q: What is machine learning?
A: Machine learning is a subset of artificial intelligence that allows systems to learn and improve from experience without being explicitly programmed. It encompasse
s supervised learning, unsupervised learning, and reinforcement learning as its main types. In supervised learning, models are trained using labeled data, while unsup
ervised learning identifies patterns in unlabeled data.
```

In [91]:

```
chat_history.extend([
    HumanMessage(content="What is machine learning"),
    AIMessage(content=result1['answer'])
])
```

In [92]:

```
chat_history
```

Out[92]:

```
[HumanMessage(content='What is machine learning', additional_kwargs={}, response_metadata={}),
 AIMessage(content='Machine learning is a subset of artificial intelligence that allows systems to learn and improve from experience without being explicitly program
med. It encompasses supervised learning, unsupervised learning, and reinforcement learning as its main types. In supervised learning, models are trained using labele
d data, while unsupervised learning identifies patterns in unlabeled data.', additional_kwargs={}, response_metadata={})]
```

In [93]:

```
## Follow up question
# Follow-up question
result2 = conversational_rag_chain.invoke({
    "chat_history": chat_history,
    "input": "What are its main types?"  # Refers to ML from previous question
})
result2
```

Out[93]:

```
{'chat_history': [HumanMessage(content='What is machine learning', additional_kwargs={}, response_metadata={}),
  AIMessage(content='Machine learning is a subset of artificial intelligence that allows systems to learn and improve from experience without being explicitly program
med. It encompasses supervised learning, unsupervised learning, and reinforcement learning as its main types. In supervised learning, models are trained using labele
d data, while unsupervised learning identifies patterns in unlabeled data.', additional_kwargs={}, response_metadata={})],
 'input': 'What are its main types?',
 'context': [Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligenc
e that enables systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised
 learning, unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds pattern
s in unlabeled data. Reinforcement learning learns through'),
  Document(metadata={'source': 'data\\doc_0.txt'}, page_content='Machine Learning Fundamentals\n\n    Machine learning is a subset of artificial intelligence that en
ables systems to learn \n    and improve from experience without being explicitly programmed. There are three main \n    types of machine learning: supervised learning
, unsupervised learning, and reinforcement \n    learning. Supervised learning uses labeled data to train models, while unsupervised \n    learning finds patterns in unla
beled data. Reinforcement learning learns through'),
  Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on a
rtificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fiel
ds like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image pro
cessing, while Recurrent Neural Networks (RNNs) and Transformers'),
  Document(metadata={'source': 'data\\doc_1.txt'}, page_content='Deep Learning and Neural Networks\n\n    Deep learning is a subset of machine learning based on a
rtificial neural networks. \n    These networks are inspired by the human brain and consist of layers of interconnected \n    nodes. Deep learning has revolutionized fiel
ds like computer vision, natural language \n    processing, and speech recognition. Convolutional Neural Networks (CNNs) are particularly \n    effective for image pro
cessing, while Recurrent Neural Networks (RNNs) and Transformers')],
 'answer': 'The main types of machine learning are supervised learning, unsupervised learning, and reinforcement learning. Supervised learning uses labeled data to tra
in models, while unsupervised learning finds patterns in unlabeled data. Reinforcement learning learns through a system of reward and punishment.'}
```

In [94]:

```
result2['answer']
```

Out[94]:

```
'The main types of machine learning are supervised learning, unsupervised learning, and reinforcement learning. Supervised learning uses labeled data to train models
, while unsupervised learning finds patterns in unlabeled data. Reinforcement learning learns through a system of reward and punishment.'
```

Using GROQ LLM's

In [97]:

```
llm
```

Out[97]:

```
ChatOpenAI(client=<openai.resources.chat.completions.completions.Completions object at 0x000001D96AA93110>, async_client=<openai.resources.chat.completio
ns.completions.AsyncCompletions object at 0x000001D96AA92FD0>, root_client=<openai.OpenAI object at 0x000001D96AB19810>, root_async_client=<openai.As
yncOpenAI object at 0x000001D96AB196E0>, model_kwargs={}, openai_api_key=SecretStr('**********'))
```

In [98]:

```
load_dotenv()
```

Out[98]:
```
True
```
In [99]:
```
os.getenv("GROQ_API_KEY")
```
Out[99]:
```
'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'
```
In [100]:
```
from langchain_groq import ChatGroq
from langchain.chat_models import init_chat_model
```
In [101]:
```
os.environ["GROQ_API_KEY"]=os.getenv("GROQ_API_KEY")
```
In [102]:
```
llm=ChatGroq(model="gemma2-9b-it",api_key=os.getenv("GROQ_API_KEY"))
llm
```
Out[102]:
```
ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x000001D9741897F0>, async_client=<groq.resources.chat.completions.AsyncCompletions object at 0x000001D974189BE0>, model_name='gemma2-9b-it', model_kwargs={}, groq_api_key=SecretStr('**********'))
```
In [103]:
```
llm=init_chat_model(model="groq:gemma2-9b-it")
llm
```
Out[103]:
```
ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x000001D9742AB110>, async_client=<groq.resources.chat.completions.AsyncCompletions object at 0x000001D9742AB610>, model_name='gemma2-9b-it', model_kwargs={}, groq_api_key=SecretStr('**********'))
```
In [ ]: